
**Estimación automática de Parámetros
de Síntesis FM
para la obtención de Timbres Musicales
mediante Redes Neuronales**

Automatic Estimation of FM Synthesis
Parameters for Sound Matching using Neural
Networks



**Trabajo de Fin de Grado
Curso 2025–2026**

Autores

David Cendejas Rodríguez
Ángel Jiménez Izquierdo

Directores

Miguel Gómez-Zamalloa Gil
Jaime Sánchez Hernández

**Grado en Ingeniería de Computadores
Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid**

**Estimación automática de Parámetros de Síntesis
FM
para la obtención de Timbres Musicales mediante
Redes Neuronales**

Automatic Estimation of FM Synthesis Parameters for
Sound Matching using Neural Networks

**Trabajo de Fin de Grado en
Ingeniería de Computadores / Ingeniería Informática**

Autores

David Cendejas Rodríguez
Ángel Jiménez Izquierdo

Directores

Miguel Gómez-Zamalloa Gil
Jaime Sánchez Hernández

Convocatoria: *Junio 2026*

**Grado en Ingeniería de Computadores
Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid**

25 de Mayo de 2026

Dedicatoria

*A Miguel y a Jaime, por su paciencia y por
habernos introducido en el mundo de la
informática musical y la investigación
académica.*

*A Carlos, por su guía en las tareas de machine
learning.*

*A nuestras familias y amigos, por su cariño y
apoyo incondicional.*

Agradecimientos

A cualquier persona involucrada en el despegue de nuestras carreras profesionales como informáticos a lo largo de estos cuatro años: profesores, compañeros, familia y amigos.

Resumen

Estimación automática de Parámetros de Síntesis FM para la obtención de Timbres Musicales mediante Redes Neuronales

La invención de los sintetizadores en los años 60 revolucionó el mundo de la música, antes dominada por los instrumentos analógicos tradicionales. La llegada de la música sintética y del ordenador como elemento central de la producción moderna han definido la forma de trabajar de los artistas modificando su sonido y procesos creativos. Desde entonces los productores musicales y diseñadores de sonido han pasado años aprendiendo a dominar los sintetizadores para definir y crear timbres musicales nunca antes vistos, sin embargo, dicho proceso a menudo requiere de mucha maestría, pues los distintos tipos de síntesis suelen ser complejos, modulares y escalables, lo que lo convierte en una tarea altamente complicada para la mayoría de los músicos. Es común que incluso los mejores diseñadores de sonido encuentren dificultad en replicar otros sonidos sintéticos, pues dependen directamente del hardware utilizado y de la secuencia de efectos y modulaciones sonoras aplicadas.

Es por ello que la aplicación del aprendizaje profundo a la síntesis abre todo un abanico de posibilidades al delegar la detección de patrones y conocimiento técnico requerido a la inteligencia artificial, permitiendo al artista ocuparse de la experimentación y la creación que le pertenece. Mediante el uso de Redes Neuronales Convolucionales (CNNs), este proyecto busca automatizar la estimación de parámetros en la compleja síntesis FM. Así, la IA resuelve la barrera técnica de replicar un sonido (*sound matching*), convirtiéndose en una herramienta directa al servicio de la creatividad.

Todo el código desarrollado se encuentra en nuestro repositorio de GitHub:
<https://github.com/Angelji06/TFG-SoundMatchingFM>

Palabras clave

Sintetizadores, Síntesis FM, Aprendizaje Profundo, Redes Neuronales Convolucionales, Procesamiento de Señales de Audio, Espectrogramas, Python, PyTorch.

Abstract

Estimación automática de Parámetros de Síntesis FM para la obtención de Timbres Musicales mediante Redes Neuronales

The invention of synthesizers in the 1960s revolutionized the music world, previously dominated by traditional analog instruments. The arrival of synthetic music and the computer as a central element of modern production have defined the way artists work, altering their sound and creative processes. Since then, music producers and sound designers have spent years learning to master synthesizers to define and create never-before-heard musical timbres; however, this process often requires a great deal of expertise, as the different types of synthesis tend to be complex, modular, and scalable, making it a highly complicated task for most musicians. It is common for even the best sound designers to encounter difficulties when trying to replicate other synthetic sounds, as they depend directly on the hardware used and the sequence of effects and sound modulations applied.

Therefore, applying deep learning to synthesis opens up a whole range of possibilities by delegating pattern detection and the required technical knowledge to artificial intelligence, allowing the artist to focus on experimentation and creation. Through the use of Convolutional Neural Networks (CNNs), this project seeks to automate parameter estimation in complex FM synthesis. Thus, AI overcomes the technical barrier of replicating a sound (*sound matching*), becoming a direct tool at the service of creativity.

All the developed code can be found in our GitHub repository:
<https://github.com/Angelji06/TFG-SoundMatchingFM>

Keywords

Synthesizers, FM Synthesis, Deep Learning, Convolutional Neural Networks, Audio Signal Processing, Spectrograms, Python, PyTorch.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	3
1.3. Plan de trabajo	3
2. Estado de la cuestión	7
2.1. Síntesis sonora y representación del audio	7
2.1.1. Síntesis sonora	7
2.1.2. Síntesis por modulación de frecuencia (FM)	8
2.1.3. Representación digital del sonido: La onda como vector	9
2.1.4. Representación espectral: STFT y espectrogramas	10
2.1.5. Escala de Mel	11
2.1.6. <i>Mel Frequency Cepstral Coefficients</i> (MFCC)	11
2.2. Inteligencia Artificial aplicada al audio	12
2.2.1. CNNs en el dominio del audio	12
2.2.2. Transformers en el dominio del audio (AST)	12
2.2.3. Inferencia automática de parámetros en síntesis	13
2.2.4. Representación del audio usada en otros proyectos	13
2.2.5. Datasets	14
3. Metodología y diseño del sistema	15
3.1. Descripción de los parámetros del sistema	15
3.2. Generación del <i>dataset</i>	16
3.3. Preprocesamiento de datos: normalizaciones y espectrogramas	17
3.4. Diseño de la arquitectura CNN	18
3.5. Funciones de pérdida	20
3.5.1. El Problema de la inyectividad	20
3.5.2. Funciones de pérdida	21
3.5.3. Modelo de pruebas simplificado	22
4. Implementación	23
4.1. Herramientas y librerías: <i>stack</i> tecnológico	23

4.2.	<i>Pipeline</i> de entrenamiento	24
4.3.	Inferencia de parámetros	25
4.4.	Interfaz gráfica de la aplicación	25
4.5.	Estructura del código	28
5.	Resultados y evaluación	31
5.1.	Definición de los pipelines experimentales	32
5.2.	Introducción y configuración del entorno	33
5.2.1.	Características del <i>hardware</i> empleado en el entrenamiento de los modelos	33
5.2.2.	Configuración de las pruebas	33
5.2.3.	Convergencia y dinámica del entrenamiento	33
5.2.4.	Análisis crítico de las métricas de evaluación	34
5.3.	Evaluación sobre el conjunto de prueba (<i>Test Set</i>)	34
5.4.	Evaluación sobre el banco de pruebas (<i>benchmark</i>)	37
5.4.1.	Comportamiento ante sonidos percusivos y cortos	38
5.4.2.	Comportamiento ante sonidos graves	39
5.4.3.	Comportamiento ante sonidos sostenidos complejos	41
5.5.	Conclusiones del capítulo	41
6.	Experimentación paralela	43
6.1.	Limitaciones de Fréchet Audio Distance (FAD)	43
6.1.1.	Descripción de la Métrica	43
6.1.2.	Limitaciones de la métrica en el contexto del <i>sound matching</i>	43
6.2.	Validación por fuerza bruta (<i>grid search</i>)	44
6.2.1.	Diseño experimental y parametrización del <i>grid search</i>	45
6.2.2.	Espectrograma de Mel	45
6.2.3.	Espectrograma MFCC	46
6.2.4.	Conclusiones	47
7.	Conclusiones y trabajo futuro	49
7.1.	Conclusiones	49
7.2.	Trabajo futuro	49
8.	Introduction	51
8.1.	Motivation	51
8.2.	Objectives	53
8.3.	Work Plan	53
	Conclusions and Future Work	55
	Contribuciones Personales	57
	Bibliografía	61

Índice de figuras

1.1. Sintetizador analógico.	1
1.2. Aphex Twin programando sintetizadores.	2
2.1. Yamaha DX7. Fuente (Hispasonic).	8
2.2. Envolvente ADSR. Fuente (djexpressions).	9
2.3. Representación del muestreo de una onda. Fuente: Wikipedia (a). . . .	10
2.4. Ejemplo de un espectrograma. Fuente: Wikipedia (b).	10
2.5. Estructura clásica de las CNN.	12
2.6. Foto del VST Dexed. Fuente: Green Musicians.	14
3.1. Representación de la estructura interna de CNNRegressorSimple. . .	19
3.2. Representación de la estructura interna de CNNRegressor5.	20
4.1. Flujo de trabajo completo del programa.	24
4.2. Pestaña principal.	26
4.3. Pestaña de entrenamiento.	27
4.4. Pestaña de pruebas.	27
4.5. Sintetizador FM propio.	28
5.1. Representación de las diferentes combinaciones de Arquitecturas y Funciones de Pérdida.	32
5.2. Comparativa global de la dispersión de las métricas Mel L1 y MCD en el Test Set para los seis pipelines experimentales.	36
5.3. Comparativa del rendimiento de los seis modelos sobre los 12 sonidos del <i>benchmark</i>	37
5.4. Espectrograma resultado de la predicción de P3 del audio de <i>Bench- mark: percusivo</i>	38
5.5. Espectrograma resultado de la predicción de P2 del audio de <i>Bench- mark: percusivo</i>	39
5.6. Espectrograma resultado de la predicción de P1 del audio de <i>Bench- mark: muy_bajo</i>	40
5.7. Espectrograma resultado de la predicción de P2 del audio de <i>Bench- mark: muy_bajo</i>	40

5.8. Espectrograma resultado de la predicción de P2 del audio de <i>Benchmark: pad_lento</i>	41
8.1. Analog synthesizer.	51
8.2. Aphex Twin programming synthesizers.	52

Índice de tablas

3.1. Rangos de los parámetros (<i>GEN_PARAMS</i>) utilizados para la generación aleatoria del <i>dataset</i>	16
4.1. Bibliotecas principales utilizadas en el proyecto.	23
5.1. Resumen del rendimiento durante la fase de entrenamiento (<i>Early Stopping</i> = 10)	33
5.2. Resultados (Media $\pm$ Desviación Estándar) de las métricas acústicas sobre el <i>Test Set</i> para las seis configuraciones.	36
5.3. Medias de las métricas acústicas sobre el <i>benchmark</i> para las seis configuraciones.	38

Introducción

“Sueño con instrumentos que obedezcan a mi pensamiento y que, con el aporte de un mundo de sonidos insospechados, se presten a las exigencias de mi ritmo interior.”

— Edgard Varèse

1.1. Motivación

La producción musical moderna ha experimentado una transformación radical en las últimas décadas, consolidando al ordenador y al *software* como los ejes centrales del proceso creativo. En este paradigma digital, la generación de señales de audio artificiales mediante sintetizadores se ha convertido en una herramienta indispensable para artistas, productores y diseñadores de sonido. Históricamente, los primeros sintetizadores analógicos, basados en métodos de síntesis aditiva¹ y subtractiva², ofrecían un flujo de trabajo relativamente accesible; su interfaz física y visual, controlada a través de potenciómetros y filtros, permitía a los músicos esculpir el sonido. A pesar de seguir siendo un proceso complejo, era algo accesible para aquel que estudiase el funcionamiento en profundidad.



Figura 1.1: Sintetizador analógico. Fuente (Dreamstime).

¹La síntesis aditiva busca la construcción de sonidos combinando (sumando, es decir, ‘por adición’) elementos más simples que el sonido original. (Hispasónic, 2026).

²Es un método de síntesis donde una señal es generada por un oscilador y después filtrada (Wikipedia, 2026).

Sin embargo, el panorama de la síntesis cambió drásticamente a finales de los años sesenta con el descubrimiento de la Síntesis por Modulación de Frecuencia (FM) y, de forma masiva, con la comercialización del emblemático Yamaha DX7 en 1983. Esta tecnología supuso una revolución en el mercado al permitir la creación de timbres metálicos, eléctricos y acampanados, acústicamente imposibles de lograr con la tecnología de la época.

No obstante, esta innovación trajo consigo un dilema significativo: su programación es notoriamente contraintuitiva. A diferencia de los sistemas tradicionales, en la síntesis FM un cambio mínimo en un solo parámetro (como el índice de modulación o la frecuencia de un operador) puede destruir por completo la estructura armónica del sonido. Esta extrema sensibilidad paramétrica y la pronunciada curva de aprendizaje han alejado históricamente a la mayoría de los creadores del diseño sonoro FM, relegándolos al uso de *presets* preconfigurados.

Desde una perspectiva matemática y acústica, este problema radica en la falta de inyectividad del motor de síntesis: combinaciones de parámetros drásticamente diferentes pueden generar resultados acústicos que el oído humano percibe como idénticos, mientras que ajustes minúsculos pueden derivar en texturas sonoras completamente opuestas. En consecuencia, el diseño manual de sonidos se convierte en una barrera técnica que interrumpe el flujo creativo del artista.

Este nivel de complejidad técnica manifiesta su mayor obstáculo en el proceso conocido como *Sound Matching* o igualación tímbrica. Es una situación cotidiana en el estudio de grabación que un productor escuche un sonido específico y desee replicarlo. Realizar esta tarea “a oído” con un sintetizador FM es un proceso de ensayo y error que puede resultar profundamente frustrante.



Figura 1.2: Aphex Twin programando sintetizadores. Fuente (Reverb Machine).

Ante esta problemática, el aprendizaje automático y, en concreto, el aprendizaje profundo (*Deep Learning*), se presentan como el puente ideal entre la abstracción creativa del músico y la complejidad matemática del sintetizador. Al delegar la inferencia de parámetros a un modelo computacional, se puede automatizar la búsqueda del sonido objetivo. Para lograrlo, el sistema debe ser capaz de “escuchar” y procesar la señal de una forma asimilable por la red. Mediante la transformación del audio unidimensional a un **espectrograma**, se logra “fotografiar” el sonido, adaptando las frecuencias a una escala logarítmica que imita fielmente la percepción del sistema

auditivo humano.

Una vez que el audio se representa como una imagen bidimensional, las CNNs emergen como la arquitectura idónea para abordar el problema. Estas redes están diseñadas para extraer características espaciales y, en el dominio del espectrograma, son excepcionalmente precisas al “ver” y clasificar las densas texturas tímbricas y armónicas del audio.

1.2. Objetivos

El objetivo principal del proyecto es desarrollar un sistema basado en redes neuronales convolucionales capaz de estimar automáticamente los parámetros de un sintetizador de modulación de frecuencia (FM) a partir de una muestra de audio, con el fin de replicar su timbre.

Como objetivos específicos, se tienen los siguientes:

- **Generación del dataset:** Diseñar e implementar un motor de síntesis capaz de generar un *dataset* masivo de audios sintéticos y sus correspondientes etiquetas (parámetros), así como su conversión a tensores.
- **Procesamiento de señal:** Establecer un *pipeline* de procesamiento de señales para transformar el audio crudo en representaciones visuales (espectrogramas) aptas para la CNN, aplicando las transformaciones y normalizaciones del dato que maximicen su rendimiento.
- **Desarrollo del modelo:** Diseñar, entrenar y optimizar varias arquitecturas de Redes Neuronales Convolucionales y funciones de pérdida en Python.
- **Evaluación psicoacústica:** Evaluar el rendimiento del modelo mediante métricas de error sobre parámetros y medidas de similitud tímbrica que simulen la percepción del oído humano.
- **Desarrollo de la interfaz:** Desarrollar un sistema con una interfaz funcional que permita acceder a todas las utilidades del proyecto de manera fácil e intuitiva.
- **Estudio de modelos:** Se estudiará y comparará el uso de distintas representaciones del espectrograma, arquitecturas de red y funciones de pérdida.

1.3. Plan de trabajo

El desarrollo de este Trabajo de Fin de Grado abarcará desde septiembre de 2025 hasta mayo de 2026. La evolución del proyecto a lo largo de estos meses se estructurará en cinco etapas claramente definidas:

- **Fase 1 (Septiembre - Octubre): Exploración y pruebas de concepto**
Hito principal: Etapa de investigación general sobre Inteligencia Artificial

aplicada a la música.

Durante estos meses, realizaremos una revisión bibliográfica para asentar las bases técnicas del aprendizaje profundo y su aplicación al audio. Desarrollaremos pruebas de concepto iniciales para evaluar las capacidades de la IA aplicada a sintetizadores, experimentando con la predicción de formas de onda y la generación de espectrogramas.

- **Fase 2 (Noviembre): Definición del proyecto, *Sound Matching* y primeros prototipos funcionales**

Hito principal: Establecimiento del *Sound Matching* FM como eje central y primer prototipo funcional.

En esta etapa, centraremos el proyecto en la Síntesis por Modulación de Frecuencia (FM) y comenzaremos con la generación de *datasets* masivos etiquetados. Entrenaremos un primer modelo de regresión CNN y diseñaremos la arquitectura del código basándonos en Programación Orientada a Objetos (POO). Además, se programará una Interfaz Gráfica (GUI) básica con el objetivo de lograr una primera prueba funcional sólida de *sound matching*.

- **Fase 3 (Diciembre - Febrero): Escalado de complejidad y métricas de evaluación**

Hito principal: Mejora de la síntesis y evaluación del modelo.

Una vez validada la base del sistema, añadiremos complejidad incorporando nuevos parámetros, como envolventes, a la generación de sonido. Esto requerirá escalar la generación del *dataset* y unificar el tratamiento de los datos para asegurar la consistencia en la obtención de los espectrogramas y su correcta normalización. Finalmente, implementaremos métricas de validación, y desarrollaremos herramientas para visualizar y evaluar los resultados del modelo.

- **Fase 4 (Febrero - Abril): Optimización arquitectónica y comienzo de la memoria**

Hito principal: Optimización profunda de la red neuronal e inicio de la redacción.

Durante estos meses nos enfocaremos en el desarrollo y refinamiento del prototipo final. A nivel de código, probaremos diferentes optimizaciones arquitectónicas en la red neuronal, como el uso de espectrogramas en escala Mel y funciones de pérdida avanzadas, implementando también un *benchmark* FM para su evaluación. En paralelo, comenzaremos la redacción de la memoria del proyecto, documentando los desarrollos previos y la base teórica fundamental (escala Mel, coeficientes MFCC, etc.).

- **Fase 5 (Abril - Mayo): Redacción final, entrenamiento de modelos definitivos y cierre**

Hito principal: Finalización de la memoria y entrenamiento de los modelos de producción.

Con el código principal estabilizado, las tareas de programación se limitarán a ajustes finales orientados a la evaluación y al registro del historial de entrenamiento. El esfuerzo principal se destinará a concluir la redacción de la memoria, elaborar los diagramas necesarios, limpiar el código y preparar los

scripts de ejecución para distintos entornos (Linux y Windows). Por último, llevaremos a cabo el entrenamiento de los modelos definitivos utilizando los recursos computacionales proporcionados por la UCM.

Capítulo 2

Estado de la cuestión

En este apartado se presenta una revisión del estado de la cuestión en relación a los temas centrales de este proyecto: la síntesis sonora, la representación digital del audio, y sus intersecciones con la Inteligencia Artificial. Se abordan tanto los fundamentos teóricos como las aplicaciones prácticas, destacando los avances más relevantes y las metodologías empleadas en investigaciones recientes.

2.1. Síntesis sonora y representación del audio

2.1.1. Síntesis sonora

La síntesis de sonido es el proceso de generar señales de audio artificialmente mediante hardware electrónico o software, sin depender de la grabación de una fuente acústica real. Las herramientas diseñadas para este propósito se denominan sintetizadores, los cuales generan sonidos en base a unos parámetros modificables.

La base de la síntesis sonora es la onda sinusoidal, descrita por la ecuación fundamental:

$$x(t) = A \sin(2\pi ft + \phi) \quad (2.1)$$

donde A representa la **amplitud** (la altura del sonido, relacionada con el volumen), f es la **frecuencia** en hercios (que determina el tono, con mayores valores produciendo sonidos más agudos), t es el **tiempo**, y ϕ es la **fase inicial** en radianes (que define el punto de inicio de la oscilación). Esta onda de seno pura es el elemento básico de toda síntesis.

El teorema de Fourier establece que cualquier forma de onda periódica, por compleja que sea, puede descomponerse en una suma de ondas sinusoidales de diferentes frecuencias, amplitudes y fases. Por esta razón, dominar la generación y manipulación de ondas sinusoidales es esencial para comprender cualquier técnica de síntesis sonora.

Históricamente, han existido diversas aproximaciones a la síntesis. Las más tradicionales incluyen las ya mencionadas síntesis aditiva y sustractiva. Estas técnicas fundamentales abrieron el camino hacia métodos matemáticamente más complejos,

como la síntesis FM.

2.1.2. Síntesis por modulación de frecuencia (FM)

La síntesis FM es un tipo de síntesis descubierta a finales de los años sesenta (Chowning, 1973). Su funcionamiento se basa en utilizar un oscilador (sistema capaz de generar una señal que se repite de forma periódica), denominado **modulador** para alterar la frecuencia de otro oscilador, denominado **portador**. El oído humano interpreta esta interacción como la creación de timbres completamente nuevos y armónicamente ricos.

Esta tecnología se popularizó a nivel mundial en 1983 con el lanzamiento del Yamaha DX7, el primer sintetizador digital de éxito comercial masivo. Fue muy relevante debido a su capacidad para generar sonidos metálicos, eléctricos y campanados, completamente novedosos para la época.



Figura 2.1: Yamaha DX7. Fuente (Hispanonic).

Como mencionamos anteriormente, la síntesis FM presenta una alta complejidad, lo que justifica la aplicación de redes neuronales para automatizar la inferencia de sus parámetros. Para entender esta complejidad, es necesario analizar la ecuación que describe su funcionamiento.

Una formulación simple de la síntesis FM se define como:

$$x(t) = A_c \sin(2\pi f_c t + I \sin(2\pi f_m t)) \quad (2.2)$$

Donde sus componentes principales son:

- A_c : Es la **amplitud del portador**, que define el volumen o ganancia de la señal.
- f_c : Es la **frecuencia portadora**, la frecuencia base del tono que se va a modular.
- f_m : Es la **frecuencia moduladora**, que determina la velocidad a la que oscila la frecuencia del portador.
- I : Es el **índice de modulación**, un factor escalar que controla cuánto se desplaza la fase del portador y, por tanto, la riqueza armónica del sonido resultante.

En nuestro proyecto se implementó una versión más compleja, usando envolventes.

Una envolvente es una función que modula un parámetro del sonido a lo largo del tiempo, permitiendo controlar su evolución desde el momento en que se activa

hasta que se apaga. En síntesis FM, las envolventes son cruciales para dar forma al carácter dinámico del sonido, afectando tanto su volumen como su textura armónica. La envolvente más común es la **ADSR** (Attack, Decay, Sustain, Release), que define cuatro fases distintas en la evolución del sonido: el ataque (el tiempo que tarda el sonido en alcanzar su máxima amplitud), el decaimiento (el tiempo que tarda en bajar a un nivel de sostenido), el sostenido (el nivel de amplitud mantenido mientras se sostiene la nota) y la liberación (el tiempo que tarda en desaparecer después de soltar la nota).

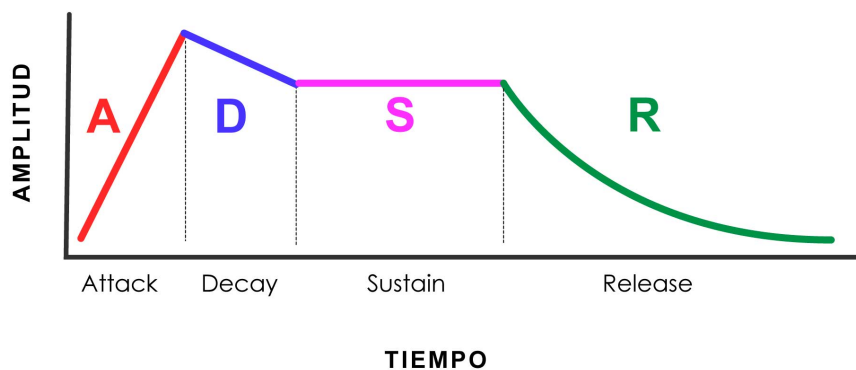


Figura 2.2: Envolvente ADSR. Fuente (djexpressions).

En nuestro proyecto se han implementado dos envolventes: una para la amplitud y otra para la modulación, ligeramente más simplificadas que la ADSR tradicional. Nuestra ecuación de síntesis FM con estas envolventes se expresa de la siguiente manera:

$$x(t) = A_{env}(t) \sin(2\pi f_c t + I \cdot M_{env}(t) \sin(2\pi f_m t)) \quad (2.3)$$

Donde:

- f_m : Se calcula dinámicamente mediante la relación portadora-moduladora (**ratio**, r), de forma que $f_m = f_c \cdot r$.
- $A_{env}(t)$: Es la **envolvente de amplitud** (**amp_env**), que sustituye a la constante A_c . Controla la ganancia global de la señal en función del tiempo según las fases de ataque (**a_att**), sostenido (**a_sus**) y decaimiento (**a_dec**).
- $M_{env}(t)$: Es la **envolvente de modulación** (**mod_env**). Escala dinámicamente el índice de modulación en el tiempo entre 0 y 1 a través de sus fases de ataque (**m_att**) y decaimiento (**m_dec**).

2.1.3. Representación digital del sonido: La onda como vector

El sonido en el mundo físico es una onda continua producida por variaciones de presión en el aire. Sin embargo, para que una máquina pueda trabajar con audio, necesita “fotografiar” esa onda miles de veces por segundo en un proceso conocido

como **muestreo** (sampling). Como resultado, la onda acústica se transforma en un vector numérico unidimensional. Cada uno de estos valores representa la amplitud de la señal en cada instante de tiempo. El número de muestras capturadas por segundo determina la calidad del audio, esto se denomina **frecuencia de muestreo** (**sample rate**).

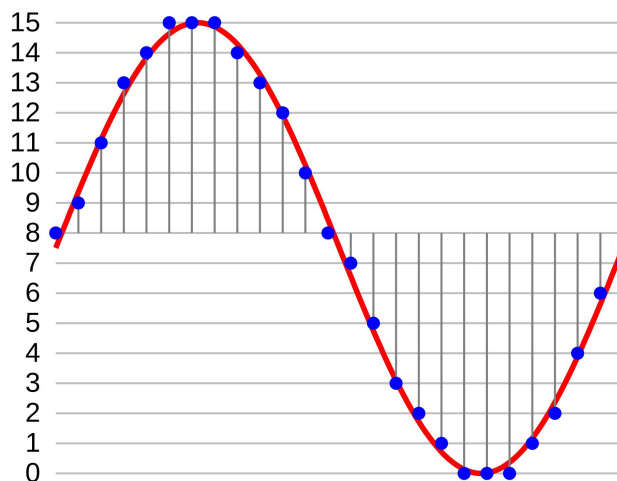


Figura 2.3: Representación del muestreo de una onda. Fuente: Wikipedia (a).

Esta representación es uno de los estándares en tareas de procesamiento de señales digitales (DSP). Sin embargo, existen otras representaciones, como el espectrograma.

2.1.4. Representación espectral: STFT y espectrogramas

El espectrograma es la representación visual del sonido más utilizada en el aprendizaje profundo, ya que permite tratar el audio como una imagen, donde el eje X es el tiempo, el Y la frecuencia y el color la intensidad. En la síntesis FM, esta representación es fundamental para que los modelos identifiquen texturas tímbricas complejas (Yee-King et al., 2018).

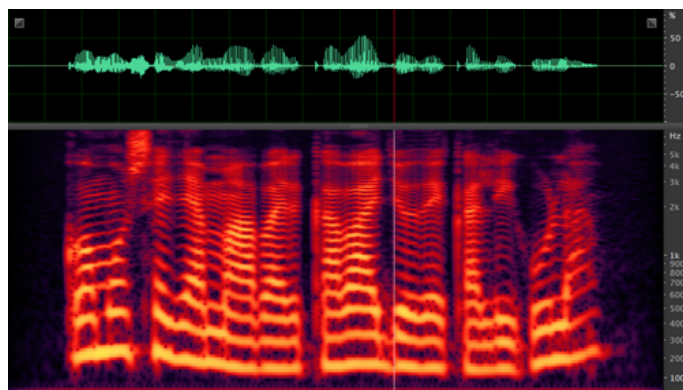


Figura 2.4: Ejemplo de un espectrograma. Fuente: Wikipedia (b).

En la parte superior de la Figura 2.4 puede verse la forma de onda del sonido, que es la representación unidimensional del vector de audio. En la parte inferior se muestra el espectrograma, que es la representación 2D del mismo sonido.

Esta imagen se genera aplicando la **Transformada de Fourier de Tiempo Reducido (STFT)** sobre el vector de audio. El proceso consiste en dividir la señal en fragmentos cortos mediante una ventana temporal y extraer el contenido frecuencial de cada uno (Allen, 1977). De este modo, se transforma una señal unidimensional en una estructura 2D que conserva la evolución temporal de las frecuencias, facilitando el análisis mediante redes neuronales convolucionales.

2.1.5. Escala de Mel

El eje de frecuencias (Y) en los espectrogramas suele tratarse como una escala lineal, donde los saltos entre frecuencias son uniformes. Sin embargo, esto puede generar una representación no intuitiva para la percepción humana, pues nosotros percibimos el sonido de forma no lineal, para ello se utiliza la **escala Mel**, la cual establece un umbral en una frecuencia específica (700 Hz), a partir de la cual la escala pasa de ser lineal a logarítmica.

Es un sistema de unidades diseñado para que distancias iguales en la escala correspondan a variaciones perceptuales uniformes en el tono (Stevens et al., 1937). Su relevancia radica en que el sistema auditivo posee una mayor capacidad de percepción en las frecuencias bajas.

La fórmula de conversión (2.4) es esencial para procesar sonidos de sintetizadores FM, los cuales generan una gran densidad de armónicos superiores que, en una escala lineal, ocuparían la mayor parte del espectro sin aportar información crítica para el oído humano.

$$m = 2595 \cdot \log_{10}(1 + f/700) \quad (2.4)$$

2.1.6. *Mel Frequency Cepstral Coefficients* (MFCC)

Los MFCC son un conjunto de coeficientes que representan la “huella” tímbrica de un sonido, es decir, aíslan el instrumento de la nota que este interpreta. Se obtienen aplicando la Transformada de Coseno Discreta (DCT) al espectrograma Mel para simplificar y organizar la información de las frecuencias (Davis y Mermelstein, 1980).

Si bien fueron el estándar en reconocimiento de voz, la tecnología actual prefiere el uso del espectrograma Mel completo. Este cambio se debe a que las redes neuronales convolucionales (CNN) logran mayor precisión al analizar la imagen sonora total, aprovechando detalles y matices que la compresión de la DCT suele eliminar en el proceso. Sin embargo, como métrica de evaluación, los MFCC siguen siendo útiles para comparar la similitud tímbrica entre el sonido original y el resintetizado.

2.2. Inteligencia Artificial aplicada al audio

2.2.1. CNNs en el dominio del audio

Las **Redes Neuronales Convolucionales (CNNs)** son un tipo de arquitectura de aprendizaje profundo diseñada para explotar la estructura espacial de los datos. Es precisamente por esto que se usan tanto en tareas de imagen. Fueron propuestas originalmente por (LeCun et al., 1998) para el reconocimiento de caracteres escritos a mano. Su principio fundamental es el uso de filtros convolucionales de pequeño tamaño (*kernels*). Dichos filtros recorren la entrada realizando productos escalares. Esto les permite detectar patrones sin importar dónde estén ubicados y usar muchos menos parámetros que una red tradicional.

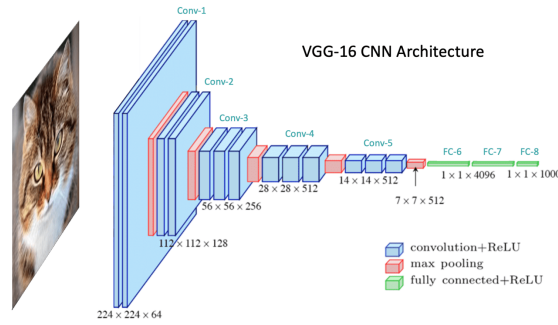


Figura 2.5: Estructura clásica de las CNN. Fuente (OpenCV).

La arquitectura típica alterna **capas convolucionales**, que extraen características locales, con **capas de pooling**, que reducen la dimensionalidad, construyendo así una jerarquía de características que va desde elementos simples, como bordes o gradientes, hacia representaciones abstractas de formas y texturas (Krizhevsky et al., 2012).

En el dominio del audio, las CNNs son eficaces porque pueden aprender patrones locales en el espectrograma, como la estructura armónica y el ruido de ataque. Se ha demostrado que modelos entrenados con grandes cantidades de datos pueden aplicar su conocimiento general a tareas específicas de síntesis (Kong et al., 2020). Al aplicar filtros convolucionales sobre las dimensiones de tiempo y frecuencia, la red identifica texturas tímbricas con una eficiencia muy superior a las redes densas tradicionales.

2.2.2. Transformers en el dominio del audio (AST)

El estado del arte en el procesamiento de audio ha evolucionado de las arquitecturas convolucionales (CNN) hacia los *Transformers*, tras su éxito en el procesamiento de lenguaje natural (Vaswani et al., 2017). En el contexto del audio, el modelo de referencia es el Audio Spectrogram Transformer (AST) (Gong et al., 2021), que adapta la arquitectura de atención para trabajar directamente con espectrogramas. A diferencia de las CNN, que analizan patrones locales mediante filtros, el AST utiliza un mecanismo de auto-atención (*self-attention*). Este permite al modelo capturar

relaciones globales y dependencias de largo alcance en el tiempo y la frecuencia, dividiendo el espectrograma en bloques de datos.

Sin embargo, los Transformers presentan una complejidad computacional cuadrática y requieren volúmenes masivos de datos para superar a las CNN. Por ello, en tareas de aprendizaje automático con recursos limitados, como es nuestro caso, las arquitecturas convolucionales siguen siendo una opción preferente por su eficiencia.

2.2.3. Inferencia automática de parámetros en síntesis

El diseño automático de sonido busca encontrar los parámetros de un sintetizador que repliquen un audio objetivo. Los primeros enfoques exitosos con redes neuronales profundas utilizaron arquitecturas LSTM y CNN para mapear sonidos a las configuraciones de operadores de un Yamaha DX7 (Yee-King et al., 2018). Sin embargo, un hallazgo crítico transformó esta metodología: optimizar el modelo basándose exclusivamente en la distancia numérica entre parámetros es ineficiente.

Este fenómeno se debe a la **falta de inyectividad** del motor de síntesis, donde combinaciones de parámetros drásticamente diferentes pueden generar resultados acústicos idénticos. Para solventar esta ambigüedad, se propuso el uso de funciones de pérdida basadas en la distancia entre los espectrogramas del audio original y el resintetizado (pérdida espectral) (Barkan et al., 2019). Esta evolución ha convergido en el uso de decodificadores avanzados, sintetizadores diferenciables como DDSP (Engel et al., 2020) y arquitecturas basadas en Transformers, que permiten una convergencia mucho más precisa pues priorizan la coherencia perceptiva del sonido sobre la coincidencia de valores numéricos.

En nuestro proyecto, las 2 funciones de pérdida que planteamos están directamente inspiradas por estos papers, una con la convergencia espectral y otra con una simulación del ventaneo de la señal que usan en el paper de DDSP.

2.2.4. Representación del audio usada en otros proyectos

El debate sobre la representación óptima divide a la comunidad entre el uso de características extraídas (como el espectrograma) y el procesamiento directo de la onda cruda (*raw audio*). En el paradigma **End-to-End (E2E)**, se elimina cualquier preprocesamiento analítico, permitiendo que la red reciba la señal unidimensional directamente. Aunque este enfoque exige una enorme capacidad computacional y de memoria debido a la extrema dimensionalidad del audio estándar (por ejemplo, 44 100 muestras por cada segundo), los modelos *End-to-End* son capaces de capturar micro-detalles temporales y relaciones de fase que inevitablemente se pierden al utilizar espectrogramas (van den Oord et al., 2016).

De nuevo, el Procesamiento de Señales Digitales Diferenciable (DDSP), permitiendo que el modelo controle osciladores y filtros directamente, es un muy buen punto medio (Engel et al., 2020).

2.2.5. Datasets

Para entrenar modelos de inferencia eficaces, se requieren bases de datos masivas y rigurosamente etiquetadas. Además, en el contexto del aprendizaje profundo aplicado al audio, la calidad de la muestra es un factor crítico; en representaciones bidimensionales como el espectrograma, la presencia de ruido o artefactos puede provocar que la CNN asimile patrones erróneos durante el entrenamiento.

En el ámbito general del análisis musical, el *dataset* NSynth se ha consolidado como el estándar actual, proporcionando más de 300 000 notas individuales de diversos instrumentos con etiquetas detalladas sobre sus cualidades acústicas (Engel et al., 2017). Sin embargo, para proyectos centrados específicamente en síntesis FM, es habitual recurrir a la generación sintética masiva utilizando emuladores por *software* como *Dexed* (un clon digital del Yamaha DX7). Este enfoque garantiza una correspondencia matemáticamente exacta entre el audio exportado y los parámetros internos del sintetizador (Yee-King et al., 2018).

No obstante, en este proyecto, con el objetivo de acotar el problema, se ha optado por desarrollar un motor de síntesis paramétrico propio, implementado íntegramente mediante la librería NumPy.



Figura 2.6: Foto del VST Dexed. Fuente: Green Musicians.

Metodología y diseño del sistema

3.1. Descripción de los parámetros del sistema

Con el objetivo de lograr un acercamiento a las aplicaciones reales del *Sound Matching* pero sin sacrificar simplicidad, se valoran cuáles deben ser los parámetros fundamentales. Finalmente se decide que el sistema trabajará con los siguientes parámetros:

- **Variables fundamentales:** Estas son la base estructural de la síntesis FM y se componen de la frecuencia portadora (*carrier*), la relación entre frecuencia portadora y frecuencia moduladora (*ratio*) y el índice de modulación (*index*). Definen el timbre del sonido, pudiendo producir desde sonidos similares al de una flauta a ruidos inarmónicos. Se decidió usar *ratio*, en vez de frecuencia moduladora, para mantener una relación armónica constante, conservando así el mismo timbre independientemente de la nota reproducida. Es decir, para que en caso de aumentar o reducir la frecuencia portadora, el sonido siga sonando igual pero en una nota diferente.

Por sí solos estos parámetros producen un sonido estático (no evoluciona en el tiempo), sonando poco orgánico. Para solucionar esto, se introducen las envolventes (descritas a continuación).

- **Envolvente de amplitud:** se han elegido las variables de *attack*, *sustain* y *decay*, evitando el *release* puesto que no aportaba demasiado al timbre y entrenar un modelo con un parámetro más implica un aumento en complejidad y tiempo significativo. Estos parámetros consiguen modelar el volumen en el tiempo, permitiendo diferenciar un sonido percusivo (con un *attack* muy rápido y sin *sustain*) de otro largo y equilibrado (de ataque lento y *sustain* duradero).
- **Envolvente de modulación:** Aquí se encuentran los parámetros de *attack* y *decay* aplicados sobre el índice de modulación. Añadiendo esta envolvente, se pueden conseguir sonidos cuyo timbre evolucione con el tiempo, siendo así más similares a instrumentos reales. Esto se debe a que al tocar un instrumento, al principio se pueden apreciar todos los armónicos perfectamente, y según el sonido decae, estos van desapareciendo con él.

3.2. Generación del *dataset*

El sistema genera un número (N) de muestras de audio con valores aleatorios, dentro de unos rangos de valores específicos para cada parámetro (ver Tabla 3.1). Para cada uno de ellos se utiliza una distribución uniforme, a excepción de la frecuencia portadora (*carrier*) que se realiza en escala logarítmica, que es similar a la percepción humana del sonido.

Se sintetizan los N archivos *.wav* mediante la librería NumPy y la formulación matemática de la FM descrita en la Sección 2.1.2. Se define una frecuencia de muestreo de 44100 Hz y establece un tiempo de duración de 2 segundos, con el fin de asegurar un margen temporal suficiente para analizar y percibir las variaciones en el sonido provocadas por las envolventes.

Parámetro	Valor mínimo	Valor máximo
Frecuencia portadora (<i>carrier</i>)	100 Hz	2000 Hz
Ratio (<i>ratio</i>)	0.05	2.0
Índice de modulación (<i>index</i>)	1	10
Ataque amplitud (<i>amp. attack</i>)	0.015 s	1.9 s
<i>Sustain</i> amplitud (<i>amp. sustain</i>)	0.015 s	1.9 s
<i>Decay</i> amplitud (<i>amp. decay</i>)	0.015 s	1.9 s
Ataque modulación (<i>mod. attack</i>)	0.01 s	1.9 s
<i>Decay</i> modulación (<i>mod. decay</i>)	0.01 s	1.9 s

Tabla 3.1: Rangos de los parámetros (GEN_PARAMS) utilizados para la generación aleatoria del *dataset*.

El modelo definitivo de generación se lleva a cabo mediante *NumPy*, de forma que todo el pipeline se mantiene unificado en un único programa python. Además, debido a la naturaleza cambiante del programa, su simplicidad nos permite modificar el algoritmo de generación rápidamente. Por ejemplo, al principio del proyecto, cuando teníamos solo 3 parámetros (*carrier*, *ratio*, *index*), aplicábamos un triple bucle for que generaba los parámetros en base a un valor de aumento. Sin embargo esta idea da problemas de *overfitting*¹ al generar una rejilla definida que el modelo acababa memorizando, incluso aplicando *jitter*² para solventarlo. Es por esto que se optó por la generación aleatoria, que aunque no es tan ordenada, es más efectiva para el aprendizaje de la red.

La generación de envolventes se lleva a cabo de manera puramente matemática con *NumPy* sobre un vector de tiempo (t). Se utiliza el mismo algoritmo geométrico para ambas envolventes, indicando un *sustain* de 0.0 en la de modulación porque carece de él.

Para el ataque, en el caso de la amplitud, se sube progresivamente el volumen de 0.0 a 1.0, y en el caso de la modulación se sube de 0.0 a 1.0 el valor que multiplica

¹Suceso que ocurre en *Machine Learning* cuando la red neuronal, en lugar de aprender patrones, "memoriza" los datos de entrenamiento.

²Técnica de inyección de ruido de forma aleatoria en los datos.

al índice de modulación. Ambos en el tiempo que va desde el segundo 0, al marcado por la variable *attack*.

En la fase de *sustain* (únicamente necesaria para la envolvente de amplitud), se mantiene el volumen fijo a 1.0 durante el tiempo indicado por la variable *sustain*.

En cuanto al *decay*, se hace lo mismo que en el ataque pero, en vez de aumentar, decreciendo linealmente de 1.0 a 0.0.

Los valores de estas envolventes se generan aleatoriamente, siendo el tiempo total máximo igual a 2 segundos. Para garantizar que no se exceda el ese tiempo, de forma que tampoco se desajusten los tiempo mínimos, se calcula el tiempo total ($t_{total} = t_{ataque} + t_{sustain} + t_{decay}$). En el caso de que sobrepase el límite de tiempo, se aplica un factor de reescalado F sobre la parte sobrante del tiempo.

$$F = \frac{T_{max} - 3 \cdot t_{min}}{t_{total} - 3 \cdot t_{min}} \quad (3.1)$$

$$t'_i = t_{min} + (t_i - t_{min}) \cdot F \quad (3.2)$$

También se genera un registro de etiquetas (*labels.csv*) que contiene los valores de los 8 parámetros en cada audio.

3.3. Preprocesamiento de datos: normalizaciones y espectrogramas

Una vez generado el dataset, este debe ser preprocesado para que la red lo pueda manejar correctamente. Así pues, los audios *.wav* se convierten a tensores nativos de *PyTorch* cargados mediante *torchaudio*. Después se le aplica una serie de transformaciones:

1. **Suavizado de amplitud:** Se les aplica un suavizado de amplitud en los extremos (*fade-in* y *fade-out*) con el objetivo de evitar chasquidos o ruidos indeseados provocados por las limitaciones de la FFT.
2. **Normalización de pico (*Peak Normalization*):** Consiste en escalar la onda de tal manera que su amplitud quede siempre acotada en el rango $[-1,1]$, consiguiendo homogeneidad en el dataset antes de generar los espectrogramas.
3. **Transformaciones principales:** Se transforma el audio a espectrograma usando STFT y después se pasa de amplitud lineal a logarítmica (dB), en ambos casos usando Librosa.

Para la representación acústica, el programa permite elegir entre dos representaciones del espectrograma, con el fin de llevar a cabo diferentes pruebas de experimentación. La diferencia reside en la escala del eje de frecuencias (eje Y en el espectrograma). Por un lado, se encuentran los espectrogramas en formato lineal, generados mediante STFT con 513 *bins*³ de frecuencia. Y por

³El término *bin* hace referencia los intervalos de la Transformada de Fourier (FFT). El tamaño de ventana utilizado para la creación de los espectrogramas es de 1024 ($N_{fft} = 1024$), lo que genera $1024/2 + 1 = 513$ intervalos o *bins* de frecuencias

el otro lado, se da lugar a elegir un formato logarítmico-perceptual (Mel con 128 *bins*), el cual tiene un mayor potencial para la representación del sonido de manera similar a la percibida por el humano.

4. **Extracción y guardado:** Posteriormente, se extrae la magnitud de la señal, se convierte a decibelios (dB) y se guarda como tensor bidimensional (formato *.pt*). Además, se genera un archivo (*spec_mode.txt*) que indica el tipo de espectrograma utilizado. Necesario para el entrenamiento, inferencia y evaluación posteriores.

Para terminar con el tratamiento del dataset, se estructuran los tensores en una clase propia que hereda de la clase Dataset de PyTorch (*SpectrogramTensorDataset*). Se lee el registro de etiquetas (*labels.csv*), y se calcula la media y desviación estándar global de cada uno de ellos. Con estas estadísticas se aplica una normalización de puntuación estándar (*Z-score*), nivelando así las diferencias entre las escalas de los parámetros, para que todos contribuyan de forma equilibrada al cálculo del error. De no aplicar esta transformación, el carrier (parámetro con valores más elevados (ej. 2000Hz)) se llevaría toda la atención de la red al calcular las pérdidas.

Finalmente, el dataset se divide, mediante una semilla fija, en: 70 % para entrenamiento, 15 % para validación (datos que la red no verá en entrenamiento, se usa para asegurar que no hay overfitting) y se guarda otro 15 % como conjunto de pruebas (*Test set*) que se usarán para las métricas finales de evaluación.

3.4. Diseño de la arquitectura CNN

Con el fin de experimentar con distintos modelos en el entrenamiento, se proponen dos arquitecturas distintas.

- **CNNRegressorSimple:** Está formada por un *Encoder* de tres bloques convolucionales, un *Bottleneck* y una cabeza de regresión pura (*Global Average Pooling*).

Primero se pasa por un Codificador (*Encoder*), el cual consta de una secuencia de tres bloques convolucionales que convierten el espectrograma inicial en una representación comprimida, extrayendo así sus características fundamentales. Para ello, cada bloque aplica núcleos (kernels) de 3x3. Se pasa por una capa de normalización por lotes (Batch Normalization), que estabiliza y acelera el entrenamiento, y una activación no lineal ReLU. Finalmente, se aplica una reducción espacial en cada bloque (Max Pooling). Con todo esto, la matriz se ha reducido a una octava parte de su tamaño original y la profundidad de la red ha incrementado de 1 a 128 canales. Como resultado, se obtiene una representación comprimida fiel al sonido original.

Al salir del *Encoder*, el flujo de datos atraviesa lo que se denomina como cuello de botella (*Bottleneck*). Este bloque convolucional no aplica una reducción espacial, manteniendo estables las dimensiones de la matriz. Su función es duplicar el número de canales (profundidad del tensor), pasando de 128 a 256

canales. Esto se hace con la finalidad de mezclar los patrones que el *Encoder* ha extraído previamente, creando una representación densa y unificada.

Después se pasa por la fase de inferencia numérica, donde ocurre la regresión de los parámetros del sintetizador FM. Se aplica una capa de agrupamiento global (*Adaptive Average Pooling*) ya que los parámetros describen el sonido completo y no un punto temporal concreto en el espectrograma. Para ello, se calcula la media de todas las posiciones de cada canal, quedándose con un único número como resumen por canal. Finalmente, el vector unidimensional, resultado del proceso anterior, atraviesa un Perceptrón Multicapa, que lo proyecta hacia las 8 neuronas de salida.

A pesar de que la red ejecuta un entrenamiento rápido, al carecer de reconstrucción no se garantiza que el *Encoder* esté extrayendo de manera acertada la estructura del espectrograma, prediciendo únicamente los valores numéricos de los parámetros.

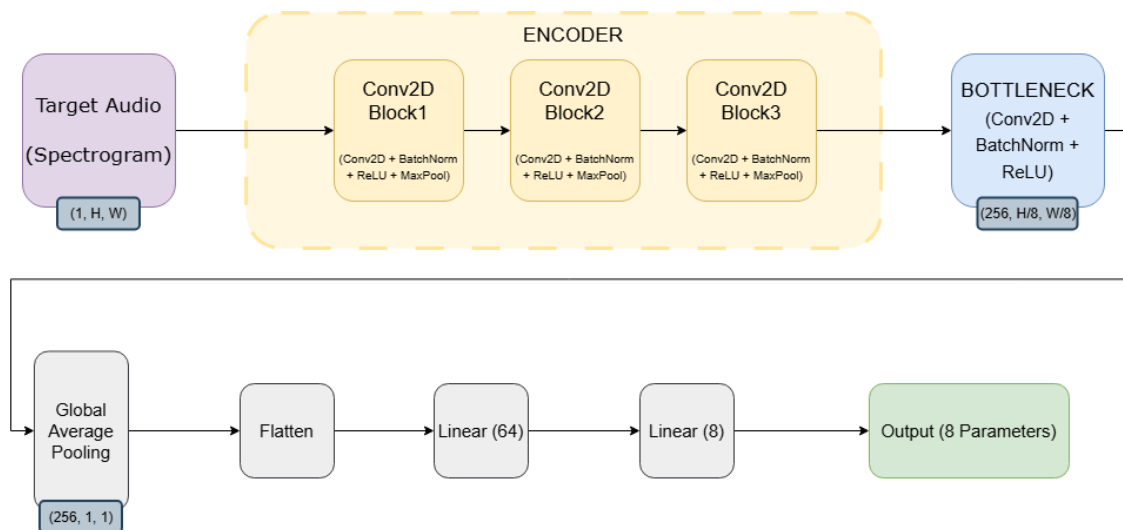


Figura 3.1: Representación de la estructura interna de `CNNRegressorSimple`.

- **CNNRegressor5:** Se trata de una expansión del modelo anterior, incorporando una segunda cabeza de predicción. Se añade la fase de Reconstrucción (*Decoder*), la cual se encarga de reconstruir el espectrograma original a partir de la representación comprimida del *Bottleneck*. Como el *Encoder* comprime tres veces, el *Decoder* descomprime tres veces. Para ello, utiliza tres capas convolucionales transpuestas (*ConvTranspose2d*).

El motivo para utilizar el *Decoder* es como método de regularización forzada del *Encoder*. En el caso en el que el codificador borre información importante, como armónicos o patrones temporales, que no afecte directamente a los parámetros de la regresión, el *Decoder* no será capaz de redibujar correctamente el espectrograma, disparando así el error en la función de pérdida. Aparte, si el *Bottleneck* no guarda suficiente información, ocurrirá lo mismo. Todo esto fuerza, mediante el cálculo de gradientes, a que las capas iniciales aprendan a conservar el timbre y la representación del sonido.

Esta representación es más costosa computacionalmente pero se estima que debería funcionar mejor que la simple.

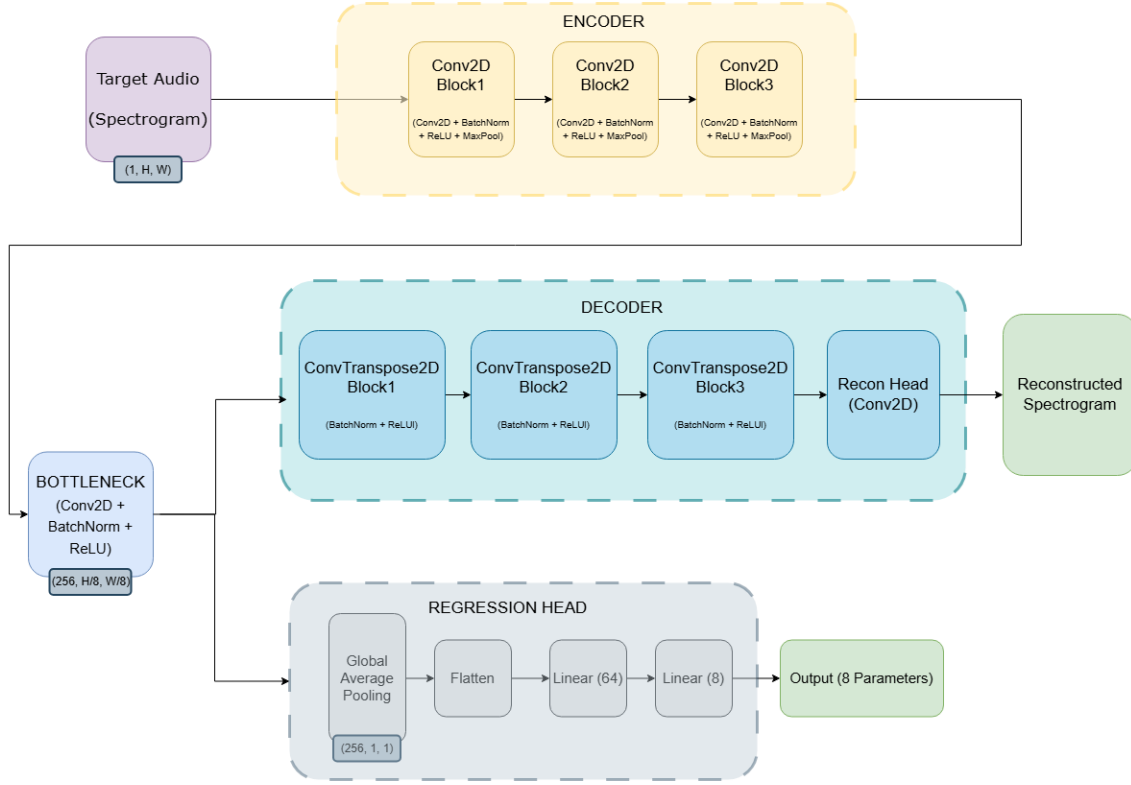


Figura 3.2: Representación de la estructura interna de CNNRegressor5.

3.5. Funciones de pérdida

3.5.1. El Problema de la inyectividad

La búsqueda de una función de pérdida se vio totalmente condicionada por una característica esencial de la síntesis FM: su inyectividad es nula. Esto quiere decir que diferentes combinaciones de parámetros pueden dar lugar a sonidos perceptualmente idénticos. Así como 2 conjuntos de parámetros muy similares entre sí pueden generar sonidos muy distintos.

Una red implementada con una función de pérdida que trate de predecir los parámetros exactos del audio original, puede verse afectada negativamente por esta característica, cosa que se comprobó en un primer prototipo cuya pérdida se media de esta forma utilizando el Error Cuadrático Medio (*MSELoss*) de los parámetros. Un ejemplo que sufría este modelo, que explica muy bien este resultado, es un caso en el que el índice de modulación sea 0. En ese caso no es verdaderamente relevante el valor del ratio, sin embargo, la función de pérdida, implementada con *MSE*, penaliza gravemente que este difiera del original.

Otro ejemplo, más común, ocurre en los casos en los que se trabaja sobre un índice de modulación alto, de tal forma que la frecuencia moduladora predomina sobre la portadora. Si tenemos un *Carrier* de 1000 Hz y un *Ratio* de 1.0, la frecuencia

moduladora resultante será de 1000 Hz, generándose los armónicos en saltos de mil en mil hercios. Por otro lado, con un *Carrier* de 2000 Hz y un *Ratio* de 0.5, se obtiene la misma moduladora de 1000 Hz y, por lo tanto, los mismos armónicos. Todo esto sumado a un índice de modulación alto, harían que estos dos audios, paramétricamente muy distantes, sonasen muy similares al oído humano.

3.5.2. Funciones de pérdida

Manteniendo el objetivo de ampliar el espacio de pruebas, se permite también el uso de diferentes funciones de pérdida. Todas basadas en aplicaciones reales en la literatura actual.

El modelo simple evalúa la diferencia paramétrica únicamente mediante *SmoothL1 Loss* (Ecuación 3.5), popularizada por el aprendizaje profundo (Girshick, 2015) e inspirada en la función de coste de Huber (1964). Esta se utiliza por su robustez y versatilidad en la regresión numérica: cuando la diferencia entre el valor real y el predicho es muy pequeña, se comporta como un Error Cuadrático (L2: Ecuación 3.4), lo que suaviza la convergencia de gradientes; y ante desviaciones grandes, se asemeja al Error Absoluto (L1: Ecuación 3.3), de forma que los casos aislados que se alejan del resto no desestabilicen la red. Esta función de pérdida tiene la desventaja de no tener en cuenta la no-inyectividad de la síntesis FM.

$$L_1(y, \hat{y}) = |y - \hat{y}| \quad (3.3)$$

$$L_2(y, \hat{y}) = (y - \hat{y})^2 \quad (3.4)$$

Siendo $\hat{y}$ una predicción y y un valor real.

$$SmoothL_1(y, \hat{y}) = \begin{cases} 0,5 \frac{(y - \hat{y})^2}{\beta} & \text{si } |y - \hat{y}| < \beta \\ |y - \hat{y}| - 0,5\beta & \text{en caso contrario} \end{cases} \quad (3.5)$$

Donde β es un hiperparámetro (umbral) que determina el punto exacto de transición entre el comportamiento cuadrático (que suaviza el gradiente) y el comportamiento lineal (que evita la desestabilización por valores atípicos). Por lo general se fija a 1.

En cuanto al modelo CNNRegressor5, se permite la elección de una de las siguientes funciones de pérdida:

- **HybridLoss:** Es una función de pérdida híbrida, que combina el Error Absoluto Medio (L1) de la reconstrucción del espectrograma, un refuerzo estructural de Convergencia Espectral (SC), y la penalización paramétrica anterior. La primera se lleva el peso principal (de 1.0), calcula el Error Absoluto Medio píxel a píxel entre el espectrograma original y el reconstruido (en decibelios). La Convergencia Espectral (SC: Ecuación 3.6) se aplica con un peso medio (de 0.5) para evitar los espectros borrosos generados por L1. Esta métrica calcula la norma de Frobenius para evaluar la energía global (relación armónica), regularizando el timbre. A la penalización paramétrica se le da una importancia muy baja (0.05), ya que no es demasiado importante debido al carácter no inyectivo de la síntesis FM.

La idea de buscar una función de pérdida que se base directamente en la diferencia de espectrogramas, y específicamente en métricas de similitud espectral como la SC, intenta ser la solución al problema de la inyectividad. El artículo *InverSynth* (Barkan et al., 2019) fue nuestra inspiración directa para implementarla, con la diferencia de que medimos el error sobre el espectrograma y no sobre los parámetros.

$$SC(S, \hat{S}) = \frac{\|S - \hat{S}\|_F}{\|S\|_F} \quad (3.6)$$

Siendo S el espectrograma original y $\hat{S}$ el generado.

- **MultiScaleSpectralLoss:** Esta función está inspirada en el modelo de síntesis diferencial DDSF (Engel et al., 2020). Al estar utilizando espectrogramas, se simulan los distintos tamaños de ventana de Fourier mediante agrupamientos espaciales temporales (*Average Pooling*). Se agrupa en seis factores (de 1, 2, 4, 8, 16 y 32) y se mide el error en cada una de esas escalas, siendo la media aritmética de esos errores el valor de pérdida total. El cálculo del error de cada escala primero se calcula la diferencia sobre las magnitudes lineales, la cual es útil para capturar picos de energía que pueden ser breves. A esto se le suma después la diferencia sobre las magnitudes en decibelios (logarítmico), multiplicada por un factor α de 1.0, por lo que se le está dando la misma relevancia que el error lineal, capturando así perfectamente la envolvente global y los momentos de baja energía. Mediante esta función, se consigue comparar el sonido obtenido del original en varios niveles de resolución simultáneamente. Por último, se le suma el error paramétrico (*SmoothL1*), de nuevo, se le otorga un peso muy bajo (0.05) con el objetivo de que la red no deje de tener mínimamente en cuenta los valores de los parámetros.

3.5.3. Modelo de pruebas simplificado

Se decide mantener un modelo simplificado, para la realización de pruebas, que infiere únicamente 3 parámetros: frecuencia portadora (*carrier*), frecuencia moduladora (*ratio*) e índice de modulación (*index*).

La topología de la red (*CNNRegressor4*) es igual a la compleja del modelo principal (*CNNRegressor5*), solo que la salida son 3 parámetros en vez de 8. La función de pérdida es la *HybridLoss* explicada en el apartado anterior.

Es el modelo usado, por ejemplo, para las pruebas de viabilidad del método *Grid Search* como sustitución de una Inteligencia Artificial.

Capítulo 4

Implementación

4.1. Herramientas y librerías: *stack* tecnológico

En la Tabla 4.1 se detallan las bibliotecas principales utilizadas en el desarrollo e implementación de este proyecto. Para cada una de ellas se especifica la versión exacta empleada, con el fin de garantizar la reproducibilidad del entorno, así como el propósito específico y las funcionalidades que aportan al sistema. En el Github del proyecto se ofrecen, además, dos instaladores de entorno virtual (Linux/Windows) para facilitar la ejecución.

Biblioteca	Versión	Dónde se usa
torch	2.5.1	Definición del modelo, capas CNN, funciones de pérdida, optimizador Adam, backpropagation y guardado de checkpoints.
torchaudio	2.5.1	Carga de WAV, transformadas Spectrogram, MelSpectrogram, AmplitudeToDB.
librosa	0.11.0	Carga de audio en inferencia, visualización de espectrogramas con eje logarítmico.
numpy	2.2.6	Síntesis FM, generación de envolventes, operaciones con arrays en evaluación y desnormalización.
sounddevice	0.5.3	Reproducción de audio en tiempo real y stream del sintetizador interactivo.
soundfile	0.13.1	Escritura de WAVs del dataset y lectura para reproducción.
matplotlib	3.10.8	Gráficas de evaluación y ventana de comparación de espectrogramas.
tkinter	8.6.13	GUI completa: páginas de inicio, entrenamiento y test; sintetizador interactivo.
ffmpeg	≥ 8.0	Backend de torchaudio para decodificación de audio.

Tabla 4.1: Bibliotecas principales utilizadas en el proyecto.

4.2. Pipeline de entrenamiento

Se adjunta la Figura 4.1 con el objetivo de facilitar la comprensión del modelo del Prototipo 5. El trabajo queda dividido en 4 fases: Generación del dataset, Entrenamiento, Evaluación e Inferencia.

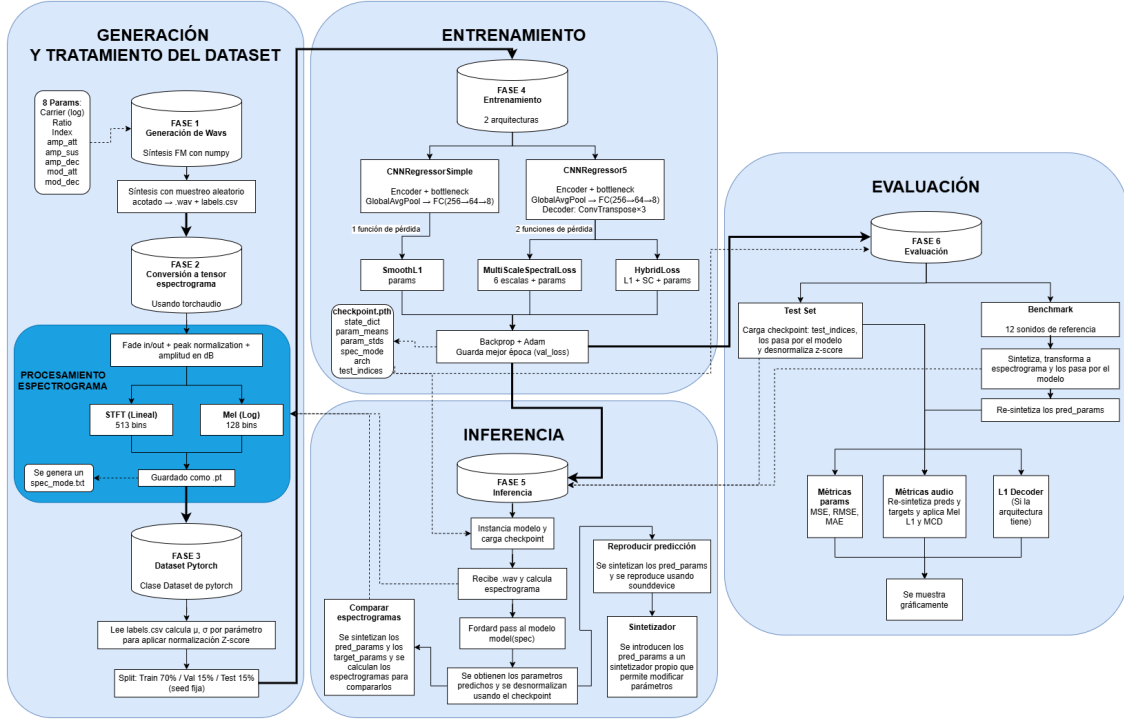


Figura 4.1: Flujo de trabajo completo del programa.

Como se muestra en dicho diagrama, en primer lugar se encuentra la generación (véase Sección 3.2) y tratamiento del dataset (véase Sección 3.3). Tras esto, el dataset ya está preparado y dividido de tal forma que se tiene un conjunto de entrenamiento que se utilizará en la siguiente fase.

La fase de entrenamiento comienza con la elección de la arquitectura y función de pérdidas (descritas en Sección 3.4 y Sección 3.5 respectivamente). Y, en la etapa final, se encuentra el algoritmo de retropropagación (*Backpropagation*), mediante el cual se actualizan los pesos para así minimizar el error en la red, calculando los gradientes del error con respecto a cada nodo. Para la actualización de los pesos se emplea el optimizador Adam (*Adaptive Moment Estimation* (Kingma y Ba, 2015)). Este se elige frente al descenso de gradiente común (SGD) debido al cálculo que hace de las tasas de aprendizaje adaptativas e independientes para cada parámetro. Aparte, se guardan los pesos de la época que ha conseguido el mínimo histórico con respecto al error sobre el conjunto de validación.

Estos valores se almacenan en un archivo de punto de control (checkpoint.pth) junto con el resto de información de contexto necesaria:

- **Parámetros de desnormalización:** Contiene las medias y desviaciones estándar obtenidas sobre los parámetros. Estas son necesarias para poder revertir

el Z-score en las futuras predicciones.

- **Contexto de arquitectura:** Son las etiquetas que describen la topología (`arch`: simple o full) y el tipo de espectrogramas (`spec_mode`: stft o mel) que se han utilizado para el modelo.
- **Test set:** Contiene los índices del conjunto de prueba (`test_indices`), necesario para la evaluación.

Una vez terminados todos estos procesos, el modelo ya se encuentra entrenado y listo para inferir parámetros o ser evaluado.

4.3. Inferencia de parámetros

Esta fase consiste en la inferencia de parámetros de síntesis FM a partir de audios, utilizando el modelo previamente entrenado. Esto significa que, una vez la red ha finalizado su aprendizaje y todos sus pesos están calibrados, es capaz de recibir nuevas muestras para realizar predicciones.

Al inicio de este proceso, se carga el punto de control (*checkpoint*), el cual instancia automáticamente la arquitectura y configuración de la red.

La predicción en sí comienza cuando se recibe un archivo de audio (*.wav*). En ese momento, se sigue el mismo flujo de preprocesamiento que en la fase de entrenamiento, transformando el audio en un tensor de espectrograma (según el tipo indicado por *spec_mode*). A continuación, este tensor se introduce en el modelo para realizar la pasada hacia adelante (*forward pass*). Es en este paso computacional donde resulta fundamental utilizar el gestor de contexto `torch.no_grad()` de PyTorch; dado que la red ya no necesita aprender ni actualizar sus pesos, desactivar el cálculo de derivadas reduce drásticamente el consumo de memoria y acelera el proceso.

Como resultado del modelo, se obtiene un vector latente de 8 valores. Sobre este vector se aplica una desnormalización (utilizando los parámetros de Z-score almacenados en el *checkpoint*), lo que da como resultado la predicción final de los parámetros de síntesis.

Llegados a este punto, el sistema permite al usuario llevar a cabo diversas acciones mediante la interfaz gráfica (GUI). Por un lado, es posible realizar una comparación visual entre el espectrograma del audio predicho y el del audio original (sintetizando ambos grupos de parámetros y calculando sus espectrogramas). Por otro lado, se ofrece la opción de comparar acústicamente ambos audios (empleando la biblioteca `sounddevice`), así como de exportar los parámetros inferidos a un sintetizador interactivo propio, donde pueden ser modificados manualmente.

4.4. Interfaz gráfica de la aplicación

Para una mejor experiencia de uso y facilitar el entorno de entrenamientos y pruebas de la aplicación, se ha desarrollado una Interfaz Gráfica. Se trata de una serie de pantallas sencillas, programadas en Python mediante el uso de la librería *Tkinter*.

En la primera pantalla (Figura 4.2), se da la opción de elegir un modelo existente o de entrenarlo.

La sección del entrenamiento (Figura 4.3) es la dedicada a seleccionar varios aspectos relevantes, tanto para el tiempo y forma de entrenamiento como para la topología del modelo. Por un lado, se permite elegir si entrenar el modelo en CPU o en GPU, lo que afectará directamente a su tiempo de entrenamiento. Por el otro, se selecciona el tipo de espectrograma, arquitectura del modelo y función de pérdida (explorados en el apartado anterior) que definirán el modelo a entrenar. El siguiente campo de la pantalla incluye la opción de crear el dataset (explicado anteriormente) o cargar uno ya generado. En último lugar se encuentra el apartado de hiperparámetros modificables para el entrenamiento, como son las épocas (*epoch*), la tasa de aprendizaje (*Learning Rate*) y los pesos de los parámetros de las funciones de pérdida.

En caso de decidir cargar un modelo, se podrá seleccionar mediante el explorador de archivos. A continuación (Figura 4.4), existen dos posibilidades: probar predicciones independientes de audios o realizar una evaluación general. En la primera, se puede cargar un wav y generar una predicción, existiendo la posibilidad de compararlos auditivamente o gráficamente sus espectrogramas (en frecuencia lineal y en logarítmica) y de ejecutar un sintetizador. En la pantalla del sintetizador (Figura 4.5), aparecen los parámetros de la predicción y los del audio original (en caso de tenerlos), siendo modificables. Además, el teclado hace la función de teclas del sintetizador con el sonido generado por los parámetros seleccionados. En la segunda, se implementan unas métricas de evaluación que se desarrollan más adelante, en el capítulo (Capítulo 5).

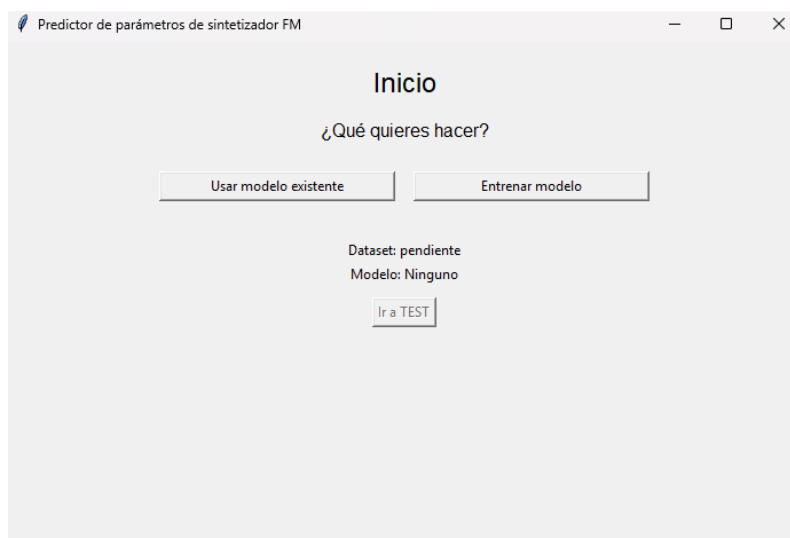


Figura 4.2: Pestaña principal.

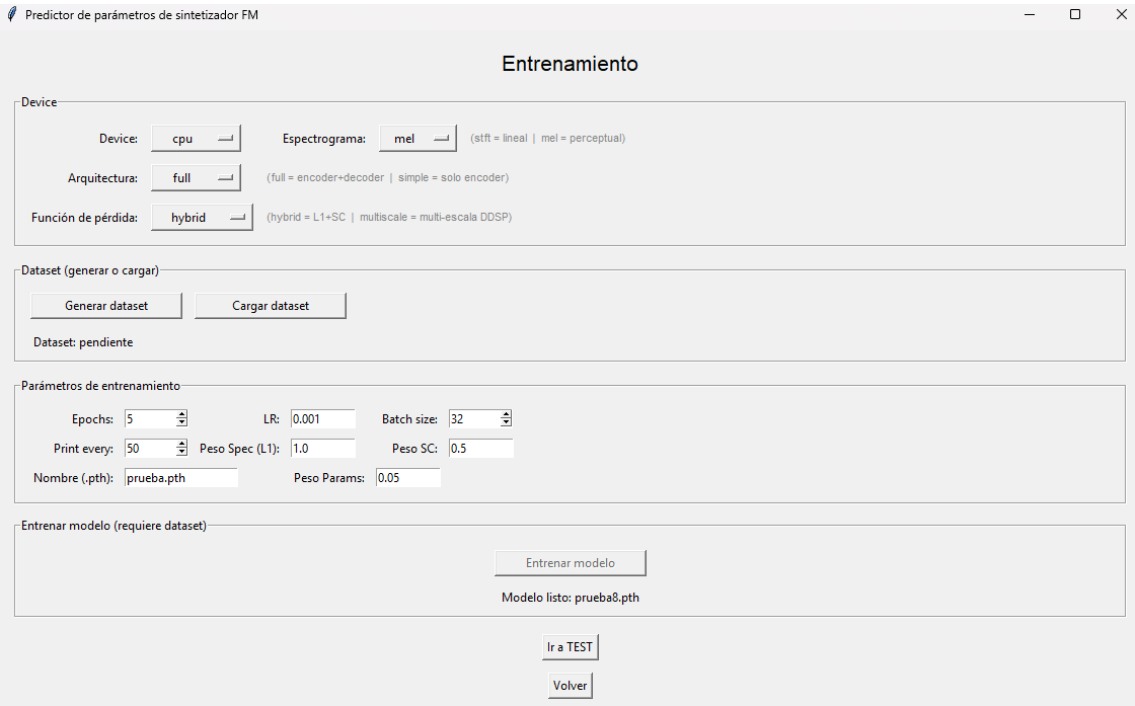


Figura 4.3: Pestaña de entrenamiento.

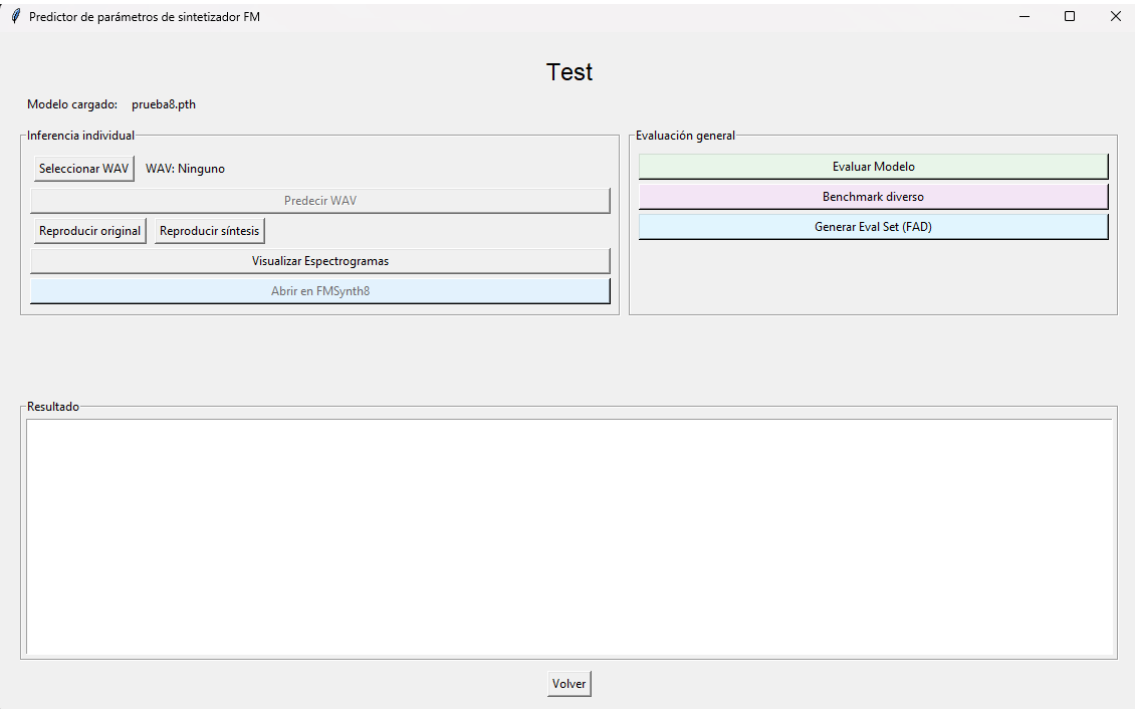


Figura 4.4: Pestaña de pruebas.

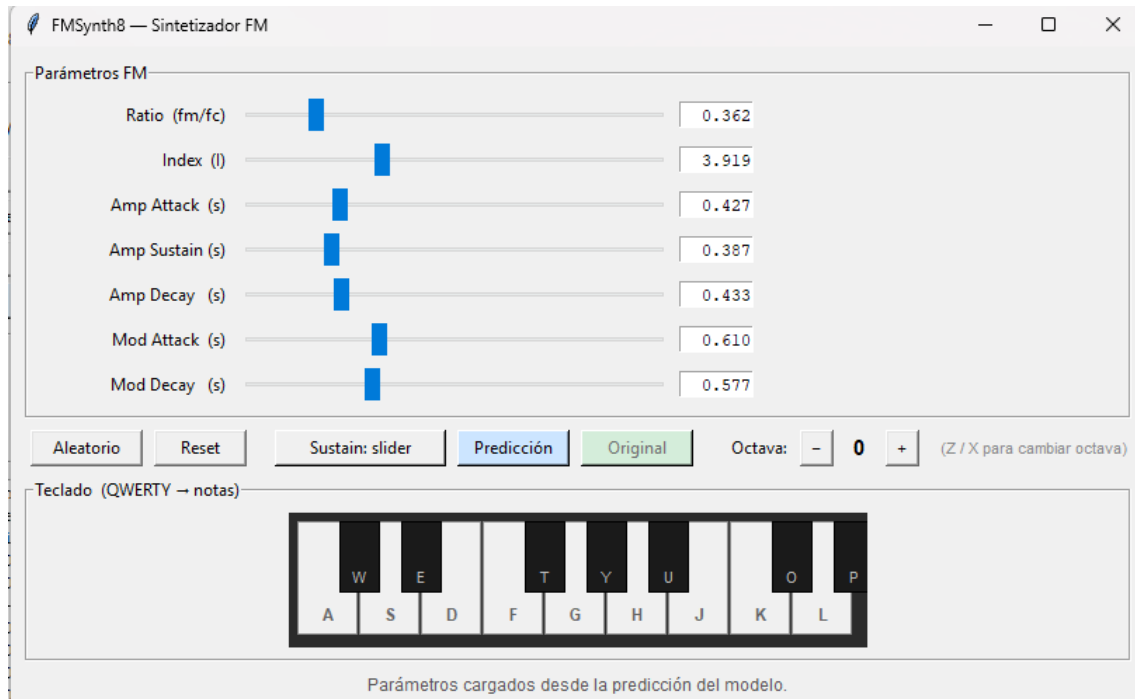


Figura 4.5: Sintetizador FM propio.

4.5. Estructura del código

Para asegurar la mantenibilidad y escalabilidad, el código se estructura mediante el principio de Separación de Responsabilidades (*Separation of Concerns*). Se divide así la aplicación en dos módulos diferentes:

- **Lógica (*backend*):** Se encuentra en el archivo `logica.py`. Aquí se agrupan las funciones matemáticas para la síntesis, cálculo de métricas, generación de espectrogramas, etc.
- **Vista y controlador (*frontend*):** Archivo `main.py`. Este implementa la interfaz, recibe los *inputs* del usuario y gestiona el flujo de llamadas a la lógica y los errores.

Adicionalmente, destaca el uso de la Programación Orientada a Objetos (POO) y el encapsulamiento. Estas características se aplican en la aplicación de la siguiente manera:

- **Clase App (`main.py`):** Se encarga de encapsular el estado global. Evita el uso de variables globales manteniendo las variables necesarias instanciadas en esta clase (`self.dataset_obj`, `self.pathModelo`, `self.model_trained`, etc).
- **Clase SpectrogramTensorDataset (`dataset.py`):** Hereda de la clase base de *Pytorch* de `torch.utils.data.Dataset`, teniendo acceso a métodos como el de procesamiento por lotes (*batching*). Asimismo, esta clase se encarga de

los cálculos de estadísticas de normalización y mapeo de los archivos, de forma que desde fuera se obtienen los datos ya preparados mediante el método `__getitem__`.

- **Modelos y funciones de pérdida (`models.py` y `losses.py`):** Como se ha mencionado previamente, existen diferentes arquitecturas y funciones de pérdida y todas ellas heredan de `nn.Modules`. En consecuencia, el código permite que la lógica del entrenamiento sea independiente del modelo exacto que se esté utilizando, lo que se conoce como **molimorfismo**. Por otro lado, la clase `HybridLoss` encapsula la lógica de la convergencia espectral y la norma de Frobenius, manteniendo el entrenamiento ajeno a estas fórmulas.
- **Sintetizador de pruebas (`FMSynth8.py`):** La pantalla del sintetizador se encuentra totalmente desacoplada del resto de la lógica, siendo una clase independiente. De esta manera, se puede utilizar tanto desde la ejecución normal de la aplicación como de forma aislada, facilitando así su construcción, ampliación y pruebas aisladas. Esta funcionalidad, debido a su complejidad a la hora de gestionar y reproducir el audio a tiempo real, ha sido íntegramente desarrollada con la ayuda de Claude Code (Anthropic (2026)).

Resultados y evaluación

En este capítulo se muestran las pruebas realizadas sobre el modelo. El objetivo es el de evaluar el rendimiento del sistema y experimentar en el uso de las diferentes combinaciones de espectrogramas, arquitecturas y funciones de pérdida para así obtener los mejores resultados posibles. Para ello, la fase se divide en dos vertientes: evaluación sobre *Test Set* y evaluación sobre *Benchmark*.

En la primera, se utilizan las muestras reservadas en la división inicial del dataset (un 15 %). Los índices exactos de dicha muestra, se obtienen del archivo de punto de control (*checkpoint*).

En cuanto a la evaluación sobre *Benchmark*, su objetivo es poner a prueba el manejo del modelo dada la característica no inyectividad de la síntesis FM. Para ello, se diseña un banco de pruebas (*benchmark*) con 12 sonidos de referencia que abarcan espacio tímbrico representativo del sintetizador (desde un bajo limpio a sonidos agudos, complejos y muy ruidosos).

Sobre los resultados obtenidos por las dos vertientes previas, se pueden calcular diferentes métricas para la evaluación del modelo en ambos ámbitos:

- **Métricas sobre parámetros:** Estas evalúan el modelo en función de la precisión obtenida en los parámetros inferidos. Se emplean las métricas de: error cuadrático medio (MSE), que penaliza las desviaciones paramétricas grandes entre predicción y original; raíz del error cuadrático medio (RMSE), que devuelve el error en las unidades originales de cada parámetro, consiguiendo ver de forma directa la precisión y así poder hacer comparaciones objetivas entre las distintas arquitecturas y funciones de pérdida; y error absoluto medio (MAE), el cual es más robusto frente a valores atípicos. Estas métricas sufren ante el problema de la no inyectividad, por lo que se utilizan principalmente como diagnóstico rápido.
- **Métricas de audio:** Estas son mucho más interesantes para conseguir una evaluación similar a la que daría un oído experto, ya que se usan para sintetizar sonidos con los parámetros obtenidos y realizan una comparativa perceptual de los sonidos. Primero se aplica la Distancia L1 en Espectrogramas Mel (*Mel l1*), la cual calcula el Error Absoluto Medio (L1) sobre las señales previamente transformadas a la escala Mel, consiguiendo así la sensibilidad logarítmica del

oído humano. De igual manera se aplica la Distorsión Cepstral Mel (MCD - *Mel-Cepstral Distortion*). Es una métrica que trata de evaluar la similitud del timbre de los audios, extrayendo 13 Coeficientes Cepstrales en las Frecuencias de Mel (MFCC). El primer coeficiente, que representa la energía global, es descartado para que la métrica refleje exclusivamente el timbre (textura del sonido).

- **Métrica de reconstrucción (*Decoder L1*):** En el caso en el que se utilice la arquitectura **CNNRegressor5**. Calcula el Error Absoluto Medio (L1) entre el espectrograma original y el reconstruido por la red, de forma que se consigue evaluar la asimilación de información en el espacio latente.

5.1. Definición de los pipelines experimentales

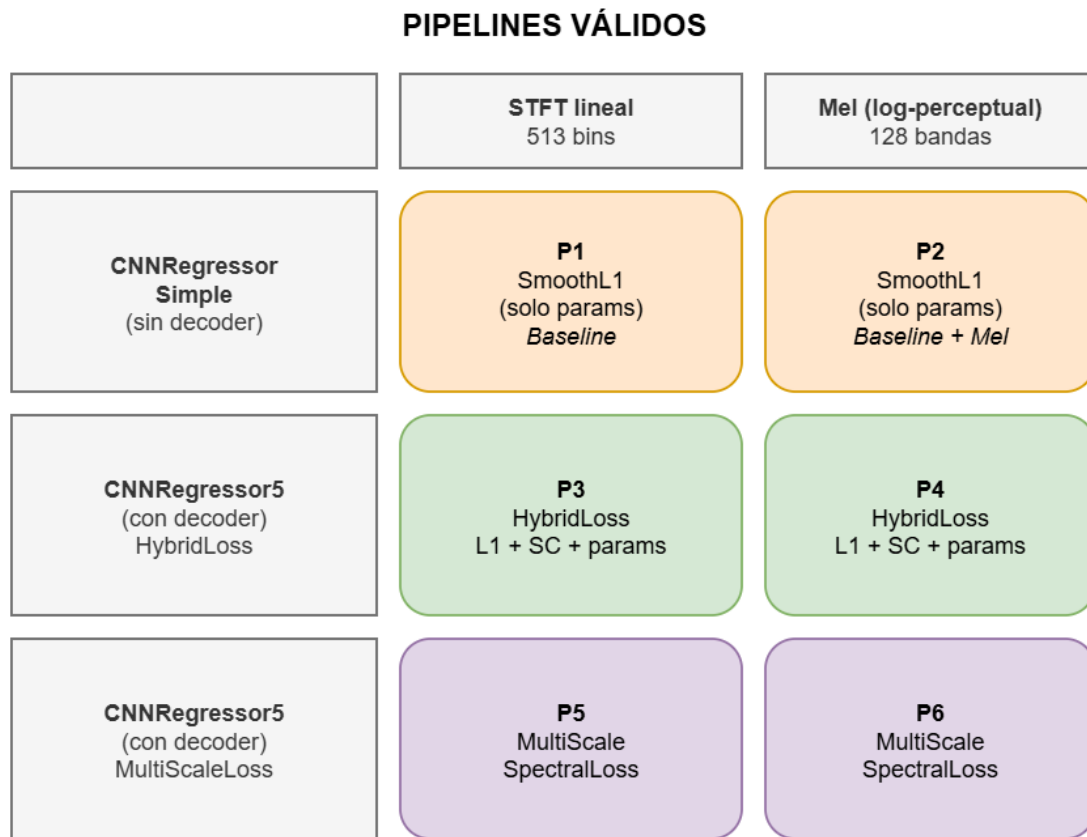


Figura 5.1: Representación de las diferentes combinaciones de Arquitecturas y Funciones de Pérdida.

La Figura 5.1 muestra las diferentes configuraciones que se dan combinando las arquitecturas y funciones de pérdida previamente definidas.

5.2. Introduccion y configuración del entorno

5.2.1. Características del *hardware* empleado en el entrenamiento de los modelos

Para la contextualización de los datos de tiempos requeridos en el entrenamiento de los diferente modelos, se especifican las características de las máquinas empleadas: **GPUs**:

- **Máquina cedida por la Universidad:** NVIDIA GeForce RTX 3070
- **Ordenador personal David:** NVIDIA GeForce RTX 3050

5.2.2. Configuración de las pruebas

La evaluación de las topologías se ha llevado a cabo con un único dataset de 50000 audios (método de generación explicado aquí: Sección 3.2), un número que ha resultado ser equilibrado, permitiendo unos resultados decentes sin suponer un incremento demasiado grande en los tiempos de entrenamiento de los modelos.

El número de épocas no se ha mantenido fijo, se ha utilizado un valor de paciencia (*Early Stopping*) de 10 épocas. De esta manera, se comprueba el error de validación en cada época y si despues de 10 seguidas no se ha mejorado su record histórico, la red deja de entrenarse. Así pues, se evita la sobreadaptación (*overfitting*) de la red neuronal, puesto que se evita un entrenamiento excesivo, y se ahorra tiempo.

Por otro lado, se ha elegido un *Batch Size* de tamaño 32. Este, aparte de ser un estándar, es un valor asumible por cualquier tarjeta gráfica moderna y que consigue una eficiencia adecuada.

Benchmark:

- **Tamaño del dataset:** 50000
- **Valor de paciencia:** 10
- **Batch size:** 32

5.2.3. Convergencia y dinámica del entrenamiento

Modelo	Función de pérdida (Validación)	Épocas	Error de validación (mínimo)
P1	SmoothL1 (Paramétrica)	35	0.0159
P2	SmoothL1 (Paramétrica)	57	0.0068
P3	HybridLoss (Acústica)	59	0.3830
P4	HybridLoss (Acústica)	25	0.6723
P5	MultiScaleSpectralLoss (Acústica)	35	0.4297
P6	MultiScaleSpectralLoss (Acústica)	60	0.6974

Tabla 5.1: . Cabe destacar que los errores de los modelos que presentan funciones de pérdida diferentes no son comparables.

La Tabla 5.1 muestra el funcionamiento de *Early Stopping* y cómo cada red tarda un número diferente de épocas en llegar a cumplir con este valor de paciencia. Destaca en la eficiencia de entrenamiento el modelo P4, el cual consigue entrenarse en tan solo 25 *epochs*, mientras que la arquitectura P3, que solo difiere en el tipo de espectrograma, necesitó extenderse hasta las 59 épocas.

Es importante remarcar que los resultados de los errores de validación de diferentes funciones de pérdida no son comparables, siendo lógico que las pérdidas paramétricas registren errores inferiores en comparación a las pérdidas acústicas, ya que estas miden diferencias de magnitud y energía (en decibelios).

5.2.4. Análisis crítico de las métricas de evaluación

Antes de continuar con las pruebas detalladas de los modelos y su análisis, es importante mencionar un sesgo en el cálculo de las métricas acústicas de error.

Durante la experimentación, se detectó una relación directa entre la duración del sonido de los audios y la puntuación obtenida sobre las métricas. En el caso de Mel L1, se calcula el error medio durante el total de la extensión de la muestra (fijada en 2 segundos) y como los modelos predicen con mucha precisión los silencios, los audios que contienen una parte considerable sin sonido consiguen puntuaciones mejores.

A pesar de este hecho, la experimentación no queda invalidada, simplemente se tiene en cuenta para la interpretación de las gráficas.

5.3. Evaluación sobre el conjunto de prueba (*Test Set*)

La Figura 5.2 recoge la distribución visual de los errores de cada arquitectura, marcando dónde se encuentra la media. Por otro lado, la Tabla 5.2 contiene la información exacta de la media y desviación estándar de cada modelo.

Al proceder con la comparación de cada modelo con su contraparte con la misma arquitectura y diferente tipo de espectrograma, se puede apreciar que los modelos con espectrogramas Mel han conseguido resultados que superan significativamente a los obtenidos con STFT. Como indican los datos, P2 consigue errores inferiores a P1, P4 a P3 y P6 a P5.

Este resultado era esperable, debido a la característica ya comentada de los espectrogramas Mel, que simulan la percepción del oído humano y facilitan la extracción de características a la red neuronal.

En cuanto al mejor resultado obtenido en la evaluación del conjunto de prueba, sorprende que se trate de P2, puesto que su función de pérdida (*SmoothL1*) es paramétrica, lo cual hemos visto con anterioridad que es arriesgado porque la síntesis FM no es inyectiva y las funciones híbridas (de P3 a P6) surgían del intento de asumir dicha característica. Adicionalmente, cabe destacar que P2 utiliza la arquitectura *CNNRegressorSimple*, de la cual, al ser más simple y carecer de *decoder*, se esperaban peores resultados.

El análisis de los resultados no nos lleva a descartar directamente los últimos modelos y a quedarnos únicamente con P2, sino que encontramos una explicación que justifica los resultados. Las estructuras simples presentan una ventaja inicial en este caso: para la red resulta considerablemente más sencillo comparar números exactos en funciones de pérdida paramétricas que evaluar imágenes (espectrogramas) en funciones acústicas.

Al añadir un *decoder* y funciones de pérdida acústicas e híbridas, es muy probable que se trate de arquitecturas muy complejas para el tamaño del dataset. En consecuencia, la red neuronal se centra también en la reconstrucción de los detalles visuales en lugar de exclusivamente en aprender los parámetros.

Como conclusión, queda claro que para el tamaño de dataset limitado que hemos utilizado, los modelos simples convergen antes en mejores resultados, pero como línea de trabajo futuro interesaría ampliar los datos y seguir probando los modelos complejos.

En cuanto a la dispersión de los valores de cada modelo (véase Figura 5.2 y Tabla 5.2), se puede apreciar que la dispersión de los modelos es, en general, elevada, sobre todo en modelos como P1 y P2 con desviaciones estándar en la métrica MCD que se acercan considerablemente a la media. En Mel L1 se suelen presentar valores que no se alejan mucho de la media pero los valores que se alejan tienen frecuencias de aparición altas, subiendo así la desviación estándar. En cuanto a MCD, la mayoría de los valores se congregan alrededor de la media, pero existen unos pocos casos con resultados muy alejados que sesgan la desviación estándar.

Para entender la escasez de consistencia de los modelos y los casos que se salen de los valores de las medias de las métricas, hace falta tener en cuenta el sesgo de Mel L1 y MCD (véase Sección 5.2.4). En la Figura 5.2, los valores de las gráficas que más se alejan de la media, es muy probable que sean audios cortos/percusivos (los que consiguen mejor puntuación) y largos/sostenidos (los que más se alejan por la derecha).

Para comprobar esta hipótesis, en la siguiente sección se analiza cómo cada red gestiona distintos tipos de audio.

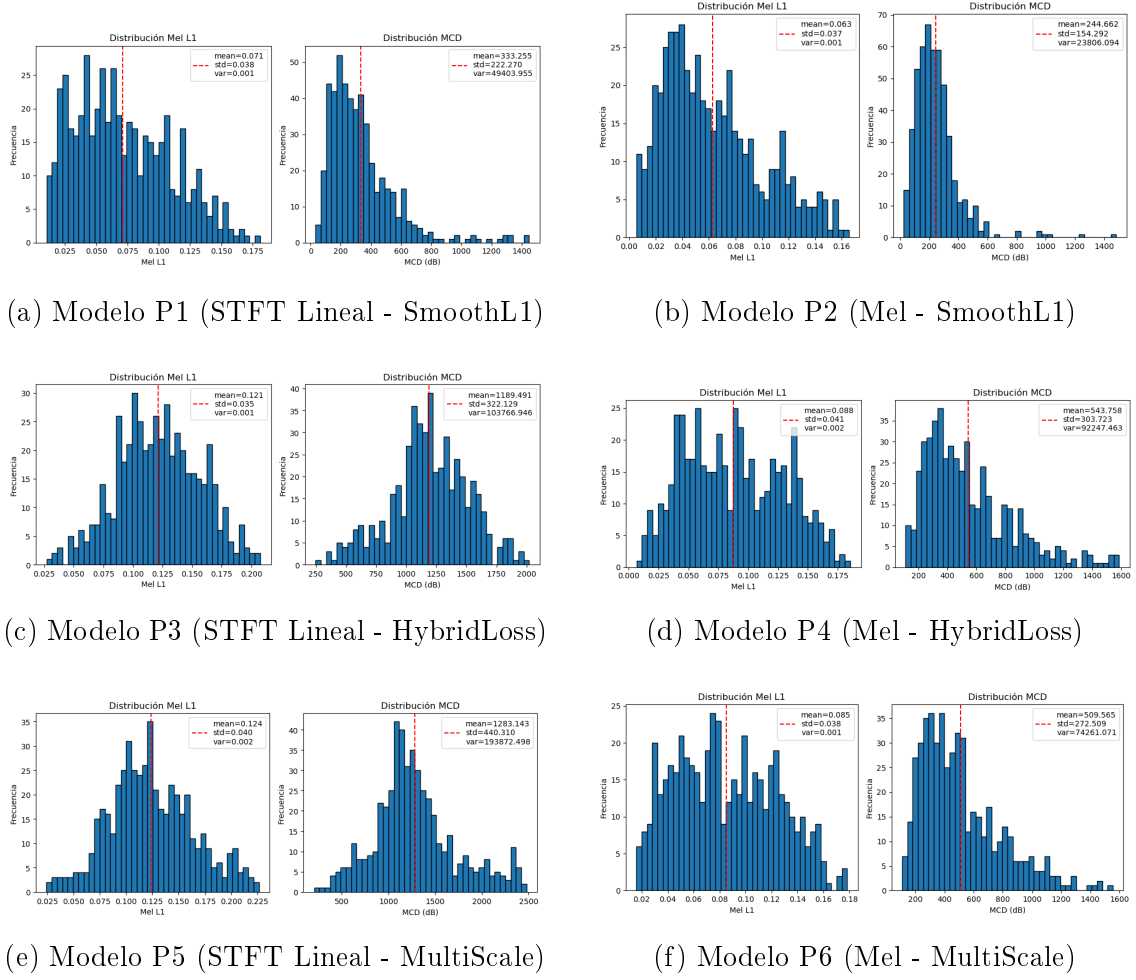


Figura 5.2: Comparativa global de la dispersión de las métricas Mel L1 y MCD en el Test Set para los seis pipelines experimentales.

Pipeline	Mel L1 (dB) ↓	MCD ↓
P1	$0,071 \pm 0.038$	$333,255 \pm 222.270$
P2	$0,063 \pm 0,037$	$244,662 \pm 154,292$
P3	$0,121 \pm 0.035$	$1189,491 \pm 322.129$
P4	$0,088 \pm 0.041$	$543,758 \pm 303.723$
P5	$0,124 \pm 0.040$	$1283,143 \pm 440.310$
P6	$0,085 \pm 0.038$	$509,565 \pm 272.509$

Tabla 5.2: Resultados (Media $\pm$ Desviación Estándar) de las métricas acústicas sobre el *Test Set* para las seis configuraciones.

5.4. Evaluación sobre el banco de pruebas (*benchmark*)



Figura 5.3: Comparativa del rendimiento de los seis modelos sobre los 12 sonidos del *benchmark*. Se evalúan las métricas Mel L1 y MCD. La media de cada una se marca con una línea roja. En el eje de horizontal se representan los distintos audios en el siguiente orden (de izquierda a derecha): 1) *bajo_limpio*, 2) *bajo_rico*, 3) *medio_armonico*, 4) *medio_inarmónico*, 5) *agudo_limpio*, 6) *agudo_ruidoso*, 7) *percusivo*, 8) *pad_lento*, 9) *pluck*, 10) *muy_bajo*, 11) *muy_agudo* y 12) *indice_maximo*.

Pipeline	Mel L1 (dB)	MCD
P1	0,042	392,724
P2	0,028	214,520
P3	0,050	702,794
P4	0,056	497,223
P5	0,076	767,566
P6	0,053	491,756

Tabla 5.3: Medias de las métricas acústicas sobre el *benchmark* para las seis configuraciones.

Se recogen los resultados de la evaluación mediante *benchmark* de los modelos en la Figura 5.3, y se mantiene la Tabla 5.3 con los valores de las medias de cada configuración.

5.4.1. Comportamiento ante sonidos percusivos y cortos

Analizando la Figura 5.3, se puede apreciar que el audio *percusivo* consigue las métricas más bajas de todos los modelos, a excepción de P5, y esto coincide con que es el audio de menor duración. Para entender esto, se observa cómo diferentes redes han gestionado su predicción:

Modelos que no habían conseguido obtener una buena media en la predicción, como es el caso de P3, consiguen un resultado muy bueno con este audio, pero si analizamos su espectrograma frente al original (Figura 5.4), se aprecia que la red ha desplazado totalmente las frecuencias generando armónicos más agudos, ha gestionado mal los tiempos y al oído suena completamente diferente. En consecuencia, se puede concluir que este audio ha sido valorado de forma positiva principalmente por su corta duración.

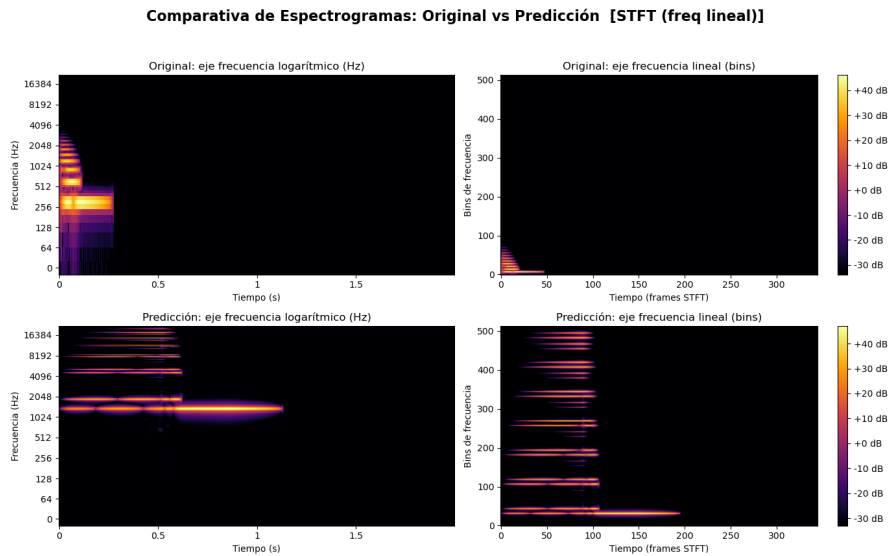


Figura 5.4: Espectrograma resultado de la predicción de P3 del audio de *Benchmark: percusivo*.

Si comparamos el resultado anterior de P3 en la Figura 5.3 con el resultado del audio *percusivo* en P2, se observa un valor similar en las métricas. Sin embargo, al analizar los espectrogramas en P2 (Figura 5.4), se aprecia una representación del audio original significativamente más similar. El modelo representa la forma perfectamente, equivocándose únicamente en la energía excesiva que otorga en frecuencias bajas. Además, al escucharlos, la predicción y el original comparten timbres considerablemente similares.

Este análisis pone en duda la robustez de las métricas para audios cortos. Por ello, se continúa analizando otros tipos de audios para así corroborar la hipótesis.

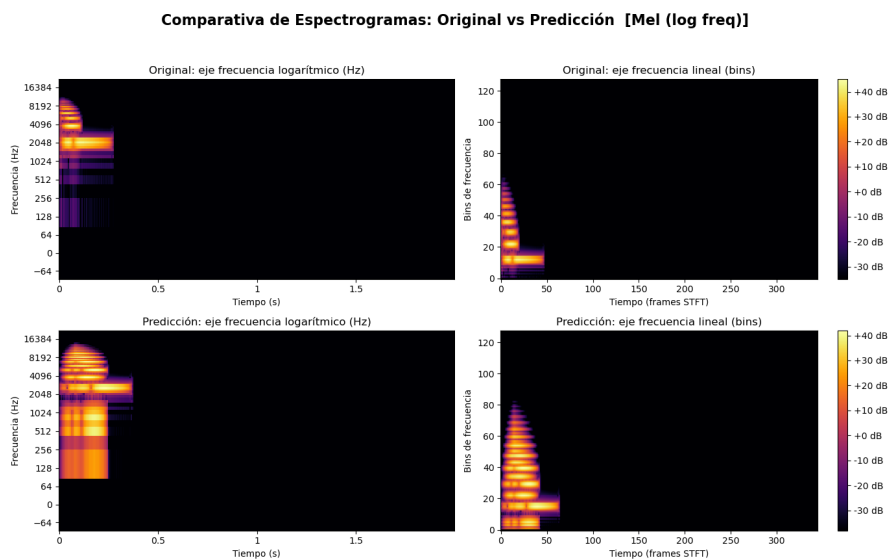


Figura 5.5: Espectrograma resultado de la predicción de P2 del audio de *Benchmark: percusivo*.

5.4.2. Comportamiento ante sonidos graves

Para este apartado es interesante explorar los resultados sobre sonidos graves, ya que, analizando las gráficas de los resultados sobre *benchmark* de audios (Figura 5.3), se aprecia que los modelos alimentados con espectrogramas STFT lineales tienen peores resultados sobre los audios graves que los que utilizan espectrogramas de Mel.

La razón de esto se puede comprobar mirando por ejemplo P1 y P2. P2 es el modelo que está consiguiendo métricas óptimas y, por lo tanto, predicciones significativamente mejores, y P1 comparte su estructura, a excepción de que utiliza espectrogramas STFT. Si comparamos los resultados obtenidos en las predicciones del audio *muy_bajo* de ambos modelos (Figura 5.6 y Figura 5.7), se puede apreciar que P2 ha conseguido una representación altamente fiel al original, mientras que P1 ha fallado en la envolvente y ha añadido frecuencias más agudas. Adicionalmente, la predicción de P2 produce un timbre más similar al original que P1.

Este hecho valida que los espectrogramas de Mel representan mejor las frecuencias bajas, las cuales son de gran interés en la generación de sonido con sintetizadores

y en la música.

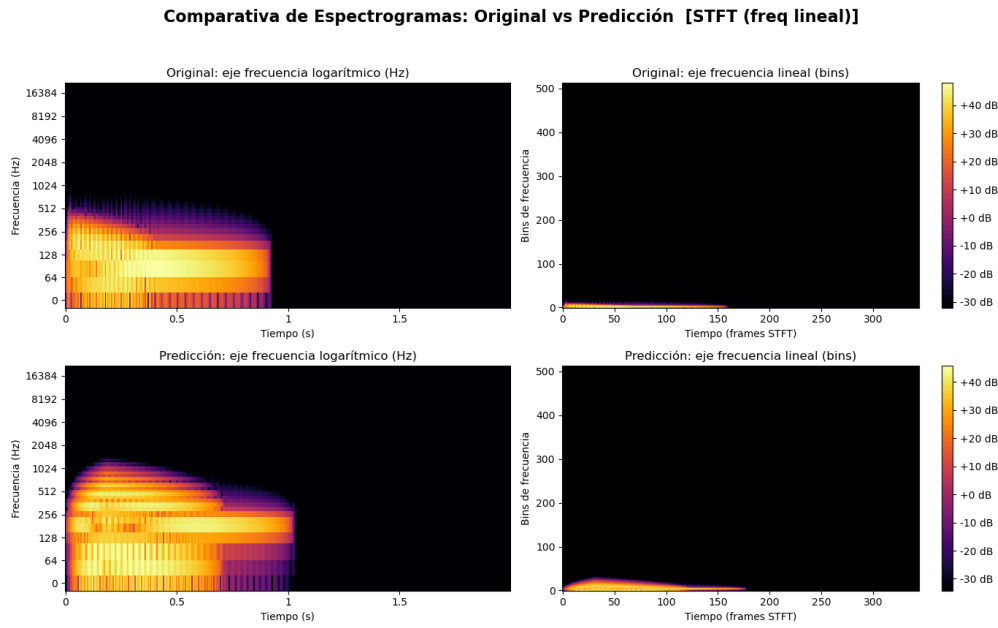


Figura 5.6: Espectrograma resultado de la predicción de P1 del audio de *Benchmark: muy_bajo*.

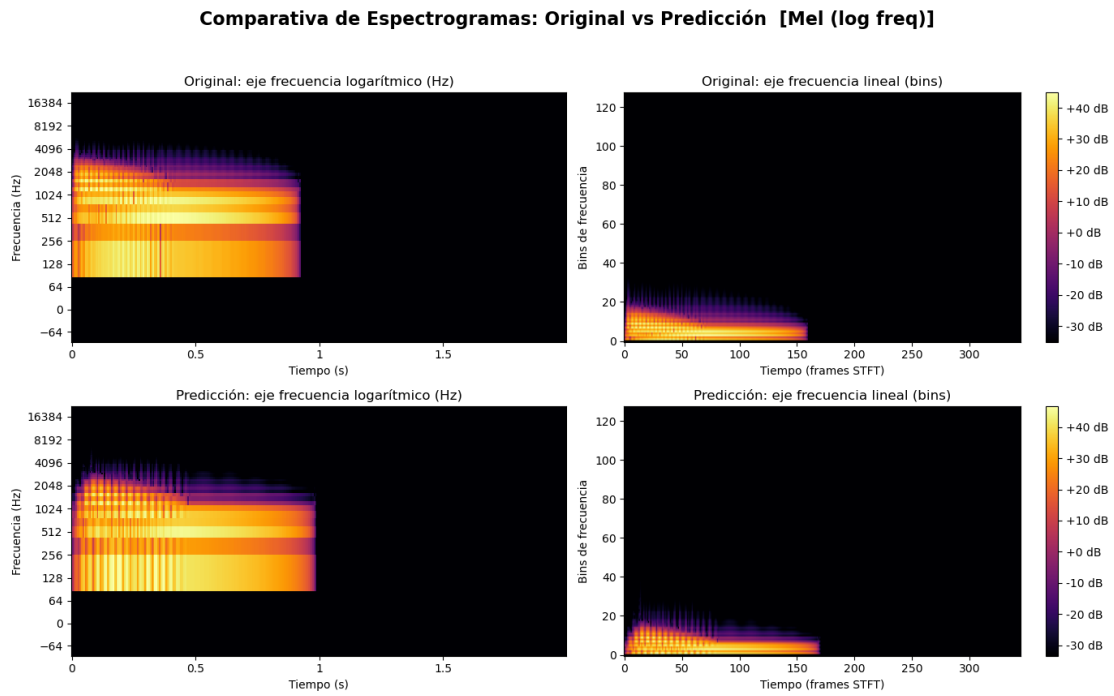


Figura 5.7: Espectrograma resultado de la predicción de P2 del audio de *Benchmark: muy_bajo*.

5.4.3. Comportamiento ante sonidos sostenidos complejos

En la Figura 5.3, se distingue el audio *pad_lento*, el cual obtiene métricas pésimas en cada uno de los modelos. Para entender lo que está ocurriendo se analiza la predicción de P2 ante dicho audio:

En el espectrograma (Figura 5.8) y en la comparación acústica, se refleja un resultado positivo, el modelo P2 consigue replicar el timbre, las envolventes y la mayoría de frecuencias a la perfección, pero falla en lo mismo que fallaba con el audio *percusivo*: aplica energía en exceso a las frecuencias bajas.

Como el audio carece de silencios y dura 2 segundos, cualquier desviación pequeña pero que se mantenga constante, eleva artificialmente la métrica.

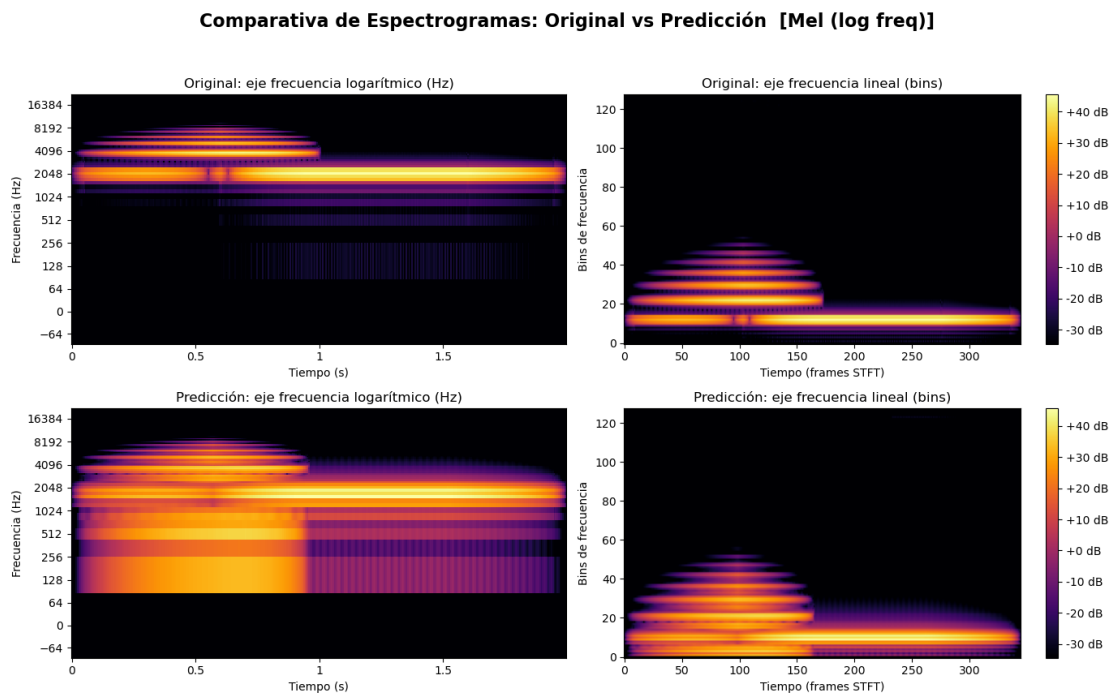


Figura 5.8: Espectrograma resultado de la predicción de P2 del audio de *Benchmark: pad_lento*.

5.5. Conclusiones del capítulo

El modelo ganador ha sido claramente P2, consiguiendo las mejores métricas y predicciones con timbres notablemente afines. Por otra parte, las configuraciones más complejas (de P3 a P6) han obtenido peores resultados debido al reducido tamaño del dataset (de 50.000 muestras) en comparación con la elevada complejidad de las arquitecturas.

En cuanto a la elección del tipo de espectrograma para la red, queda claro que es una decisión considerablemente relevante y que afecta directamente a los resultados, demostrándose que los espectrogramas de Mel representan las frecuencias más relevantes para el oído humano y, por lo tanto, para el proyecto.

Por último, queda empíricamente probado que las métricas Mel L1 y MCD, a pesar de representar de manera correcta la similitud acústica de los sonidos, no son fiables para comparar audios de distintas duraciones temporales (véanse Sección 5.4.1 y Sección 5.4.3). En consecuencia, se propone para futuros proyectos diseñar una métrica que no dependa de la duración de los audios.

Capítulo 6

Experimentación paralela

En este capítulo se abordan ciertas pruebas paralelas a la experimentación principal que han sido de gran valor para descartar opciones alternativas a las finalmente implementadas.

6.1. Limitaciones de Fréchet Audio Distance (FAD)

6.1.1. Descripción de la Métrica

La métrica *Fréchet Audio Distance* (FAD) (Kilgour et al., 2019) fue introducida originalmente por investigadores de Google AI y es utilizada actualmente en proyectos muy grandes como MusicLM (Andrea Agostinelli, 2023), pudiendo considerarse como el estado del arte para medir la calidad objetiva de los audios generados mediante Inteligencia Artificial, adaptando al dominio del audio las métricas utilizadas en la generación de imágenes.

Conceptualmente, el FAD deriva de *Fréchet Inception Distance* (FID), una métrica referente en la evaluación de modelos generativos de imágenes. Al igual que su predecesora, esta métrica no realiza comparación muestra a muestra, sino que compara características de alto nivel extraídas por una red neuronal profunda.

Para la extracción de estas características, el FAD utiliza un modelo llamado *VGGish*, un clasificador de audio entrenado con una base de datos de gran envergadura de vídeos de YouTube. Este modelo transforma la señal de audio en *log-mel features* extraídas del espectrograma de magnitud.

Finalmente, la métrica se obtiene calculando la distancia de *Fréchet* entre la música de referencia y la generada. Un valor de FAD reducido indica una estadística similar entre el audio de referencia y el generado.

6.1.2. Limitaciones de la métrica en el contexto del *sound matching*

La métrica FAD fue la primera que se seleccionó como forma de evaluación del modelo. Su implementación en el proyecto, se realiza a través de la comparación de

dos conjuntos de datos: una muestra de 2.000 audios FM generados aleatoriamente y sus predicciones con el modelo de pruebas simplificado (véase Sección 3.5.3). El resultado obtenido fue de 0.9999, indicando una similitud estadística casi absoluta entre las distribuciones de audio.

A pesar de que el resultado obtenido plantea un funcionamiento cercano a la perfección, su análisis y el del funcionamiento de FAD indican una discrepancia. El motivo de la poca validez de la métrica en el proyecto, se atribuye a un error en la interpretación de su utilidad real, ya que el FAD está diseñado para evaluar la calidad global de modelos generativos, mientras que este proyecto se centra en la estimación de los parámetros que luego utiliza para generar audio FM. Puesto que tanto el conjunto de referencia como las predicciones se crean mediante síntesis FM, su huella espectral y sus características de alto nivel (*embeddings*) son casi idénticas. Por tanto, el FAD evalúa positivamente que el modelo ha aprendido a recrear el timbre o "textura" de la síntesis FM, pero, como se comentaba previamente, resulta que no lo hace muestra a muestra (audio original con audio predicho). En resumen, se prioriza la verosimilitud estadística del sonido sobre la similitud de las muestras una a una.

6.2. Validación por fuerza bruta (*grid search*)

Una de las cuestiones principales que se plantean en el desarrollo de este proyecto es si el uso de un modelo de Inteligencia Artificial representa la aproximación óptima para el objetivo de la estimación de parámetros de síntesis FM. La disyuntiva de explorar otras opciones surge, sobre todo, al analizar los primeros prototipos, como el **Prototipo 4** (modelo de pruebas simplificado), en el que se emplean únicamente 3 parámetros para la síntesis FM: *carrier*, *index* y *ratio*. Debido al reducido tamaño de este espacio de parámetros, resulta lógico plantear en la viabilidad de realizar un método de búsqueda exhaustiva (*grid search*). Este enfoque consiste en realizar un recorrido iterativo por todas las combinaciones posibles, evaluándolas y registrando aquellas configuraciones que presenten una mayor similitud acústica con respecto al sonido objetivo.

Por otro lado, surge la necesidad de encontrar un método robusto para evaluar la fidelidad del modelo de inferencia de parámetros, buscando una métrica que simule la percepción de un oído humano experto haría un oído humano experto. Esta necesidad y la cuestión anterior convergen en la fase de experimentación. De este modo, se consiguen probar diferentes métricas que evalúen la similitud acústica dentro del algoritmo de *grid search*, comprobando simultáneamente la viabilidad técnica del enfoque de fuerza bruta frente al modelo predictivo.

Para lograr esta simulación del sistema auditivo humano, se decide utilizar la escala Mel y MFCC (véase Sección 2.1.5 y Sección 2.1.6) para calcular las métricas en las pruebas.

6.2.1. Diseño experimental y parametrización del *grid search*

El espacio de parámetros del sintetizador FM seleccionado para el algoritmo de *grid search*, buscando un balance entre resolución de la búsqueda y coste computacional, fue el siguiente:

- **Carrier (frecuencia portadora):** Rango de 100 Hz a 2000 Hz, con incrementos de 100 Hz (20 valores).
- **Ratio (relación de la frecuencia moduladora y la portadora):** Rango de 0.05 a 2.0, con incrementos de 0.05 (40 valores).
- **Index (índice de modulación):** Rango de 1.0 a 10.0, con incrementos de 0.5 (19 valores).

Esta configuración genera 15.200 combinaciones únicas. Por cada iteración del triple bucle, el algoritmo crea un audio de un segundo de duración mediante síntesis FM. A continuación, se calcula el Error Cuadrático Medio (MSE) respecto a las características del audio original.

Para poner en contexto los datos de tiempo, se especifica que los algoritmos fueron ejecutados en CPU en un ordenador con procesador *Ryzen 5 5500*.

6.2.2. Espectrograma de Mel

La mayor parte de los resultados ejecutados con la métrica Mel en *grid search*, muestran valores que difieren significativamente de los originales. Al contar con un espacio de rejilla de tamaño considerable, el algoritmo encuentra mínimos locales en puntos diferentes a los valores originales, ya que estos no existen en el espacio de posibles soluciones.

Un ejemplo claro de este comportamiento se observa al analizar una inferencia donde los parámetros objetivo eran *carrier* = 1055,54 Hz, *ratio* = 0,704 e *index* = 9,38. Los resultados de los tres mejores candidatos del *grid search* fueron los siguientes:

```
=====
Tiempo total de ejecución: 1 minutos y 5.16 segundos
=====
```

```
#1 -> Error Mel: 113.6648 | Carrier: 400.0, Ratio: 1.85, Index: 9.5
#2 -> Error Mel: 126.5552 | Carrier: 400.0, Ratio: 1.85, Index: 9.0
#3 -> Error Mel: 148.8820 | Carrier: 400.0, Ratio: 1.85, Index: 10.0
```

Aunque la diferencia entre original y predicción en la frecuencia portadora (*carrier*) es superior a 600 Hz, el cálculo de la frecuencia moduladora latente ($f_m = \text{carrier} \times \text{ratio}$) explica esta selección. En el audio objetivo, la frecuencia moduladora es de $1055,54 \times 0,704 \approx 743,07$ Hz y, en la combinación seleccionada por el algoritmo es de $400,0 \times 1,85 = 740,0$ Hz.

Al operar con un índice de modulación elevado ($index > 9$), la frecuencia portadora deja de ser tan relevante, trasladando el peso acústico a las bandas laterales generadas por la modulación. El algoritmo de fuerza bruta, incapaz de seleccionar el tono original por la limitación de la rejilla, optó por anclar el parámetro *carrier* en 400 Hz para forzar un *ratio* que replicara con precisión la frecuencia moduladora original (740 Hz). Esto valida empíricamente que, sin un espacio de inferencia continuo como el que proporciona el modelo predictivo de IA, las métricas espectrales falsifican los parámetros encontrando atajos que los acerquen al sonido objetivo, si éxito en gran parte de las ocasiones.

El siguiente ejemplo muestra qué ocurre si esta vez se utiliza un *carrier* que el algoritmo no pueda seleccionar por la limitación de la rejilla (saltos de 100 Hz) y un índice de modulación bajo.

Parámetros originales:

```
Carrier = 1752.58
Ratio = 0.62
Index = 1.03
```

La aproximación del algoritmo fue la siguiente:

```
=====
Tiempo total de ejecución: 1 minutos y 4.39 segundos
=====

#1 -> Error Mel: 176.1064 | Carrier: 1600.0, Ratio: 0.70, Index: 2.0
#2 -> Error Mel: 177.4902 | Carrier: 1600.0, Ratio: 0.70, Index: 1.5
#3 -> Error Mel: 189.0954 | Carrier: 1700.0, Ratio: 0.65, Index: 1.0
```

En este caso, el algoritmo deja en tercera posición a la opción con los valores más parecidos a la original. Mientras que la tercera opción, al oído resulta muy similar pero desafinada, la primera opción difiere excesivamente.

Matemáticamente, al incrementar artificialmente el *index*, el algoritmo aumenta la anchura de banda del espectro, dispersando la energía de tal forma que parte se solapa con la frecuencia, objetivo (1752 Hz), minimizando así el MSE. Sin embargo, acústicamente, el resultado es totalmente diferente: el algoritmo ha sacrificado un tono puro y definido por un sonido rico en armónicos y percusivo, únicamente para evadir la penalización de un mínimo local inaccesible.

6.2.3. Espectrograma MFCC

En contraposición a lo observado con el Espectrograma Mel, la evaluación mediante los MFCC obtiene unos resultados lo más cercanos posibles a los valores iniciales de los parámetros (teniendo en cuenta que la anchura de la rejilla impide que sean más exactos).

Por esta razón, el audio obtenido queda, en la mayoría de ocasiones, notablemente desafinado con respecto al original. Sin embargo, se consigue plasmar la *textura* (o

timbre) con mucha similitud, debido a la precisión en la inferencia de los parámetros de *ratio* e *index*.

Los siguientes resultados son los obtenidos con los mismo audios que los utilizados de ejemplo en Mel.

Parámetros originales:

```
Carrier = 1055.54
Ratio = 0.704
Index = 9.38
```

Aproximación del algoritmo:

```
=====
Tiempo total de ejecución: 1 minutos y 10.58 segundos
=====
```

```
#1 -> Error MFCC: 124.7339 | Carrier: 1100.0, Ratio: 0.70, Index: 9.0
#2 -> Error MFCC: 146.7065 | Carrier: 1100.0, Ratio: 0.70, Index: 8.5
#3 -> Error MFCC: 171.0587 | Carrier: 1100.0, Ratio: 0.65, Index: 9.5
```

Parámetros originales:

```
Carrier = 1752.58
Ratio = 0.62
Index = 1.03
```

La aproximación del algoritmo fue la siguiente:

```
=====
Tiempo total de ejecución: 1 minutos y 9.93 segundos
=====
```

```
#1 -> Error MFCC: 121.4236 | Carrier: 1700.0, Ratio: 0.70, Index: 1.0
#2 -> Error MFCC: 143.8489 | Carrier: 1700.0, Ratio: 0.65, Index: 1.0
#3 -> Error MFCC: 155.1412 | Carrier: 1800.0, Ratio: 0.65, Index: 1.5
```

En estos resultados se puede apreciar lo explicado previamente, el algoritmo encuentra los valores más cercanos a los originales dentro de los saltos de la rejilla. Aun así, cabe mencionar que MFCC no es tan sensible a los cambios de *pitch* como lo es Mel, lo que implica que, aun teniendo unos saltos minúsculos en el *carrier*, MFCC no conseguiría en todas las ocasiones la nota exacta.

6.2.4. Conclusiones

El análisis de los datos obtenidos en estas pruebas ha servido, por un lado, para responder a las cuestiones iniciales y, por otro, ha planteado nuevas ideas y preguntas.

En cuanto a la viabilidad del método *grid search* para la inferencia de parámetros de un sintetizador FM, se demuestra con contundencia que los tiempos de ejecución (que aparecen en los resultados anteriores) de dicho algoritmo son órdenes de magnitud superiores a los obtenidos en la síntesis realizada por el modelo predictivo. Teniendo el primero una media aproximada de 1 minuto y 5 segundos para Mel y 1 minuto y 10 segundos para MFCC, frente a los 2 o 3 segundos que tarda el modelo de inferencia. A esto se le añade el hecho de que se está utilizando un tamaño de rejilla bajo de únicamente 15200 iteraciones, si se ampliasen estos números los tiempos aumentarían también.

Aparte de los tiempos de ejecución, cabe destacar la precisión. Mientras que el método *grid search* realiza una búsqueda discreta de valores, que está sujeta al tamaño de rejilla, la red neuronal es capaz de operar en un espacio continuo.

Conclusiones y trabajo futuro

Este capítulo está dedicado a la agrupación de conclusiones extraídas a lo largo del proyecto y a explorar las futuras líneas de trabajo al alcance.

7.1. Conclusiones

A través de este proyecto se ha demostrado la viabilidad de utilizar CNNs para automatizar la estimación de parámetros en la síntesis FM. El sistema desarrollado logra reducir la barrera técnica del *sound matching*, confirmando que el aprendizaje profundo es una herramienta eficaz para agilizar el flujo de trabajo de productores y diseñadores de sonido, permitiéndoles centrarse en el proceso creativo.

Esto se ha corroborado comparándolo con modelos tradicionales como el *grid search*, demostrando así su superioridad en eficiencia y viabilidad computacional (véase Sección 6.2).

Adicionalmente, con la experimentación realizada para el Capítulo 5, se han contrastado diferentes opciones de arquitecturas y configuraciones, obteniendo resultados inesperados con los que se ha podido concluir que las arquitecturas complejas no garantizan un aumento del rendimiento ya que requieren de un gran entrenamiento y tamaño del *dataset*.

Por último, cabe destacar la importancia que ha tenido en el proyecto la aplicación de interfaces gráficas y de la automatización del *pipeline*, mediante las cuales ha sido posible gestionar distintas arquitecturas, entrenamientos y pruebas de forma sencilla.

7.2. Trabajo futuro

Un trabajo de esta naturaleza es fácilmente continuable desde múltiples perspectivas; sin embargo, consideramos relevante destacar las cuatro ramas de investigación y desarrollo más prometedoras:

1. **Implementación de un sintetizador diferenciable:** La integración de librerías como DDSP (*Differentiable Digital Signal Processing*) permitiría inte-

grar el proceso de síntesis directamente en el entrenamiento de la red. Esto posibilitaría un cálculo exacto en el descenso de gradiente (al no tratar el sintetizador como una caja negra"), optimizando la convergencia del modelo y mejorando notablemente la fidelidad de los resultados.

2. **Escalabilidad a otros métodos de síntesis:** El flujo de trabajo establecido sienta las bases para expandir el sistema más allá de síntesis por modulación de frecuencia. Sería de gran interés adaptar el modelo para estimar parámetros en síntesis aditiva, sustractiva, de modelado físico o granular.
3. **Exploración arquitectónica y optimización:** Para superar los límites de rendimiento actuales, se propone un estudio más exhaustivo de hiperparámetros, la evaluación de arquitecturas de red alternativas y el diseño de funciones de pérdida personalizadas. Esto permitiría aportar enfoques propios y novedosos, reduciendo la dependencia de los modelos estandarizados en la literatura actual.
4. **Desarrollo de un plugin VST:** De cara a su viabilidad comercial y práctica, el paso lógico es encapsular el modelo en un formato VST para su integración directa en cualquier DAW (*Digital Audio Workstation*). Esto requerirá el diseño de una interfaz gráfica de usuario (GUI) intuitiva y la programación de un sistema más robusto para el manejo de excepciones, evitando que la aplicación falle ante entradas imprevistas o casos límite (*edge cases*) por parte del usuario final.
5. **Evaluación perceptual mediante pruebas de escucha:** Aunque el rendimiento del modelo se ha medido utilizando métricas cuantitativas, la incorporación de una validación cualitativa a través de usuarios reales supondría un avance significativo. La realización de pruebas de percepción subjetiva con productores y diseñadores de sonido permitiría evaluar la similitud tímbrica desde una perspectiva psicoacústica real, aportando una métrica de éxito empírica y directamente alineada con el oído humano.

Introduction

“I dream of instruments obedient to my thought and which, with the contribution of a whole world of unsuspected sounds, will lend themselves to the exigencies of my inner rhythm.”

— Edgard Varèse

8.1. Motivation

Modern music production has undergone a radical transformation in recent decades, consolidating the computer and software as the central axes of the creative process. In this digital paradigm, the generation of artificial audio signals through synthesizers has become an indispensable tool for artists, producers, and sound designers. Historically, the first analog synthesizers, based on additive synthesis¹ and subtractive synthesis² methods, offered a relatively accessible workflow; their physical and visual interface, controlled via potentiometers and filters, allowed musicians to sculpt the sound. Although it remained a complex process, it was accessible to anyone who studied its operation in depth.



Figure 8.1: Analog synthesizer. Source (Dreamstime).

However, the synthesis landscape changed drastically in the late 1960s with the

¹Additive synthesis seeks to construct sounds by combining (adding, that is, 'by addition') simpler elements than the original sound. (Hispasónica, 2026).

²It is a synthesis method where a signal is generated by an oscillator and then filtered (Wikipedia, 2026).

discovery of Frequency Modulation (FM) Synthesis and, on a massive scale, with the commercialization of the iconic Yamaha DX7 in 1983. This technology represented a revolution in the market by allowing the creation of metallic, electric, and bell-like timbres that were acoustically impossible to achieve with the technology of the time.

Nevertheless, this innovation brought about a significant dilemma: its programming is notoriously counterintuitive. Unlike traditional systems, in FM synthesis a minimal change in a single parameter (such as the modulation index or the frequency of an operator) can completely destroy the harmonic structure of the sound. This extreme parametric sensitivity and the steep learning curve have historically alienated most creators from FM sound design, relegating them to using preconfigured presets.

From a mathematical and acoustic perspective, this problem lies in the lack of injectivity of the synthesis engine: drastically different parameter combinations can generate acoustic results that the human ear perceives as identical, while minuscule adjustments can result in completely opposite sound textures. Consequently, manual sound design becomes a technical barrier that interrupts the artist's creative flow.

This level of technical complexity presents its greatest obstacle in the process known as *Sound Matching* or timbral matching. It is a daily occurrence in the recording studio for a producer to hear a specific sound and wish to replicate it. Performing this task "by ear" with an FM synthesizer is a process of trial and error that can be deeply frustrating.



Figure 8.2: Aphex Twin programming synthesizers. Source (Reverb Machine).

Faced with this problem, machine learning, and specifically Deep Learning, present themselves as the ideal bridge between the musician's creative abstraction and the synthesizer's mathematical complexity. By delegating parameter inference to a computational model, the search for the target sound can be automated. To achieve this, the system must be capable of "listening to" and processing the signal in a way that the network can assimilate. By transforming the one-dimensional audio into a **spectrogram**, the sound is effectively "photographed," adapting the frequencies to a logarithmic scale that closely mimics the human auditory system's perception.

Once the audio is represented as a two-dimensional image, CNNs emerge as the ideal architecture to approach the problem. These networks are designed to

extract spatial features and, in the spectrogram domain, are exceptionally precise at "seeing" and classifying the dense timbral and harmonic textures of audio.

8.2. Objectives

The main objective of the project is to develop a system based on convolutional neural networks capable of automatically estimating the parameters of a frequency modulation (FM) synthesizer from an audio sample, in order to replicate its timbre.

As specific objectives, the following are established:

- **Dataset generation:** Design and implement a synthesis engine capable of generating a massive dataset of synthetic audios and their corresponding labels (parameters), as well as converting them into tensors.
- **Signal processing:** Establish a signal processing pipeline to transform raw audio into visual representations (spectrograms) suitable for the CNN, applying data transformations and normalizations that maximize its performance.
- **Model development:** Design, train, and optimize various Convolutional Neural Network architectures and loss functions in Python.
- **Psychoacoustic evaluation:** Evaluate the model's performance using parameter error metrics and timbral similarity measures that simulate the perception of the human ear.
- **Interface development:** Develop a system with a functional interface that allows access to all project utilities easily and intuitively.
- **Model study:** Study and compare the use of different spectrogram representations, network architectures, and loss functions.

8.3. Work Plan

The development of this Bachelor's Thesis will span from September 2025 to May 2026. The evolution of the project throughout these months will be structured into five clearly defined stages:

- **Phase 1 (September - October): Exploration and proof of concept**
Main milestone: General research stage on Artificial Intelligence applied to music.
During these months, a literature review will be conducted to establish the technical foundations of deep learning and its application to audio. Initial proofs of concept will be developed to evaluate the capabilities of AI applied to synthesizers, experimenting with waveform prediction and spectrogram generation.

- **Phase 2 (November): Project definition, *Sound Matching*, and first functional prototypes**

Main milestone: Establishment of FM *Sound Matching* as the central axis and the first functional prototype.

In this stage, the project will focus on Frequency Modulation (FM) Synthesis and begin generating massive labeled datasets. A first CNN regression model will be trained, and the code architecture will be designed based on Object-Oriented Programming (OOP). Additionally, a basic Graphical User Interface (GUI) will be programmed with the aim of achieving a solid first functional proof of sound matching.

- **Phase 3 (December - February): Scaling complexity and evaluation metrics**

Main milestone: Synthesis improvement and model evaluation.

Once the baseline system is validated, complexity will be added by incorporating new parameters, such as envelopes, into the sound generation. This will require scaling dataset generation and unifying data treatment to ensure consistency in obtaining spectrograms and their proper normalization. Finally, validation metrics will be implemented, and tools to visualize and evaluate the model's results will be developed.

- **Phase 4 (February - April): Architectural optimization and report drafting kickoff**

Main milestone: Deep optimization of the neural network and start of writing.

During these months, the focus will be on developing and refining the final prototype. At the code level, different architectural optimizations in the neural network will be tested, such as using Mel-scale spectrograms and advanced loss functions, while also implementing an FM benchmark for evaluation. In parallel, the writing of the project report will begin, documenting previous developments and the fundamental theoretical basis (Mel scale, MFCC coefficients, etc.).

- **Phase 5 (April - May): Final drafting, definitive model training, and closure**

Main milestone: Completion of the report and training of production models.

With the main code stabilized, programming tasks will be limited to final adjustments oriented toward evaluation and training history logging. The main effort will be allocated to concluding the writing of the report, creating the necessary diagrams, cleaning the code, and preparing execution scripts for different environments (Linux and Windows). Finally, the definitive models will be trained using the computational resources provided by the UCM.

Conclusions and Future Work

Through this project, the feasibility of using CNNs to automate parameter estimation in FM synthesis has been demonstrated. The developed system succeeds in reducing the technical barrier of *sound matching*, confirming that deep learning is an effective tool to streamline the workflow of producers and sound designers, allowing them to focus on the creative process.

A work of this nature can be easily continued from multiple perspectives; however, we consider it relevant to highlight the five most promising branches of research and development:

1. **Implementation of a differentiable synthesizer:** The integration of libraries such as DDSP (*Differentiable Digital Signal Processing*) would allow the synthesis process to be integrated directly into the network training. This would enable an exact calculation in gradient descent (by not treating the synthesizer as a "black box"), optimizing model convergence and significantly improving the fidelity of the results.
2. **Scalability to other synthesis methods:** The established workflow lays the groundwork for expanding the system beyond frequency modulation synthesis. It would be of great interest to adapt the model to estimate parameters in additive, subtractive, physical modeling, or granular synthesis.
3. **Architectural exploration and optimization:** To overcome current performance limits, a more exhaustive study of hyperparameters, the evaluation of alternative network architectures, and the design of custom loss functions are proposed. This would allow for the contribution of novel, proprietary approaches, reducing the reliance on standardized models in current literature.
4. **Development of a VST plugin:** Looking toward its commercial and practical viability, the logical step is to encapsulate the model in a VST format for direct integration into any DAW (*Digital Audio Workstation*). This will require the design of an intuitive graphical user interface (GUI) and the programming of a more robust system for exception handling, preventing the application from crashing due to unforeseen inputs or *edge cases* from the end user.

5. **Perceptual evaluation through listening tests:** Although the model's performance has been measured using quantitative metrics, incorporating qualitative validation through real users would represent a significant advancement. Conducting subjective perception tests with producers and sound designers would allow for the evaluation of timbral similarity from a real psychoacoustic perspective, providing an empirical success metric directly aligned with human hearing.

Contribuciones Personales

David Cendejas Rodríguez

Mis principales contribuciones a este trabajo se han centrado en el liderazgo del desarrollo técnico, la investigación de arquitecturas de Deep learning y la implementación de los diversos prototipos que han dado forma al sistema final, el prototipo 5. Mi labor ha abarcado desde el planteamiento arquitectónico inicial hasta el entrenamiento y validación de los modelos, asumiendo la responsabilidad de investigar las tecnologías de Machine Learning y Deep Learning aplicadas al ámbito del audio. Este proceso ha supuesto un reto constante, especialmente al abordar la síntesis FM, debido a problemas intrínsecos como la nula inyectividad y la extrema sensibilidad de sus parámetros, factores que han condicionado directamente la convergencia y el rendimiento de nuestro modelo de Sound Matching.

Debido a ello, también he sido el principal responsable del desarrollo del contenido del Resumen, Introducción, Estado de la Cuestión y Conclusiones y Trabajo Futuro de la memoria, además he realizado los diagramas necesarios para las partes más técnicas.

Paralelamente, he llevado a cabo una exploración técnica profunda sobre las representaciones de los datos y las estrategias de normalización. Esta parte del trabajo ha sido fundamental para transformar señales de audio en estructuras de datos procesables, como tensores espectrograma, asegurando la estabilidad del entrenamiento. En este sentido, diseñé e implementé pipelines robustos para el preprocesamiento de audio, garantizando que la entrada al modelo fuera óptima para el aprendizaje de las características tímbricas y unificarlo a los procesos de inferencia y evaluación, permitiendo que el sistema se comportara y escalara de forma coherente en todas sus fases del desarrollo.

En lo relativo a la ingeniería de software, he sido el principal responsable de la implementación del código en Python, aplicando principios de Programación Orientada a Objetos para garantizar un sistema modular y escalable. Debido a la naturaleza investigadora y no lineal del proyecto, el código evolucionó de forma orgánica tras cada reunión con los tutores y la revisión de literatura técnica. Esto me exigió realizar constantes tareas de abstracción y refactorización para evitar la redundancia y mantener la encapsulación, asegurando siempre la eficiencia computacional y la legibilidad del software. Asimismo, gestioné íntegramente las fases de depuración y

manejo de errores, lo que resultó crucial para asegurar la integridad de los resultados obtenidos en un entorno de desarrollo tan dinámico.

Finalmente, al disponer del hardware con mayor capacidad de cómputo del equipo, y a su vez el miembro que se conectó vía ssh a las máquinas proporcionadas por la universidad, asumí la responsabilidad técnica del entrenamiento de los modelos usados por mi compañero para realizar las pruebas y comparaciones.

Ángel Jiménez Izquierdo

Durante el desarrollo del proyecto, los aspectos en los que he aportado han sido principalmente como encargado de validación experimental, responsable del análisis de los rendimientos y como principal redactor de la memoria, produciendo también código relevante en diferentes aspectos de la aplicación final.

En las fases iniciales del proyecto, diseñé pruebas de concepto como la primera generación de datos (*dataset*) y los espectrogramas iniciales. Estas fueron unas implementaciones fundamentales para comenzar con la comparación de métodos de representación y generación del *dataset* y que han servido como base para los siguientes prototipos.

Por otro lado, integré el uso de las interfaces gráficas iniciales como método de entrenamiento, prueba y visualización de diferentes aspectos de los modelos, participando también en su evolución hasta la aplicación final. Esto fue de gran utilidad para mejorar la experiencia de uso, acercándonos más a un producto final y usable, permitiéndonos ser más eficientes entrenando los modelos, generando predicciones y, en definitiva, gestionando la aplicación desde la interfaz en lugar de tener que ir ejecutando *scripts* individuales. Adicionalmente, automaticé la gestión de los primeros entornos virtuales de librerías diseñados por mi compañero, centralizándolos y facilitando su uso.

A nivel de desarrollo de *Deep Learning*, diseñé e implementé el bucle de validación en entrenamiento que se ha mantenido hasta el modelo final. De esta forma, proporcioné la lógica necesaria para su monitorización, capturando la información sobre los pesos de la red.

En lo que a evaluación de los modelos se refiere, realicé las primeras pruebas de concepto, implementando un *pipeline* para la evaluación mediante el método FAD (*Fréchet Audio Distance*). Para ello, programé funciones necesarias en la generación de paquetes de tamaño configurable y para producir sus predicciones, debido a que FAD trabaja con distribuciones grandes de datos. Esta prueba permitió descartar una métrica reconocida como estado del arte en la evaluación del audio generado mediante IA y sentar las bases de cómo se desarrollarían el resto de pruebas de evaluación en la interfaz gráfica.

En la etapa previa al modelo final, exploré una vía experimental con el objetivo de mejorar significativamente los resultados procesando directamente las inferencias. Los modelos frecuentemente generaban predicciones con timbre similar pero distinto tono (*pitch*), así que decidí implementar que la aplicación ofreciera varios resultados con distintos tonos cercanos al original. Sin embargo, decidí descartar esta idea posteriormente porque no tenía en cuenta el índice de modulación, haciendo que no

funcionase correctamente en todos los casos.

Como método de validación del uso de *Deep Learning* para el propósito del proyecto, decidí llevar a cabo una prueba para su comparación con un algoritmo de búsqueda exhaustiva (*Grid Search*). Como resultado, conseguí justificar empíricamente la necesidad de usar redes neuronales para este propósito y su eficiencia, aportando respaldo práctico a la idea inicial del proyecto. Además, en este mismo proceso, extraje conclusiones sobre cómo diferentes formas de representar los audios podían influir directamente en el resultado obtenido por los modelos. Todo esto lo representé más tarde en el capítulo 6 de la memoria.

Para facilitar las explicaciones y justificar la provisión de recursos computacionales (GPUs) en reuniones con profesionales de la materia en la facultad, generé documentación técnica de las características de los modelos y de las pruebas que habíamos llevado a cabo. Adicionalmente, estos informes sentaron las bases estructurales para comenzar con la redacción de la memoria del proyecto, adaptándolos y ampliándolos, resultando así en los primeros borradores.

Con el objetivo de facilitar la ejecución del entrenamiento de los modelos por parte del profesor de la facultad encargado de ello, mi compañero redactó los pasos necesarios y yo me encargué de probarlos en un entorno virtualizado Linux (Ubuntu) para así comprobar su correcto funcionamiento. De esta manera, detecté problemas que me surgieron en esta tarea y pude dejarlos por escrito en las instrucciones. A pesar de esto, finalmente nos dieron acceso a las máquinas, por lo que no se utilizaron pero quedan registradas y pueden ser de gran utilidad para futuras implementaciones, asegurando la portabilidad multiplataforma.

Como principal redactor de la memoria, me he encargado de la redacción de los capítulos 3, 4, 5 y 6. Para la traducción rigurosa de código a texto de la metodología y el diseño del sistema en el capítulo 3, tuve que llevar a cabo una documentación exhaustiva y una recopilación profunda de las justificaciones de cada decisión técnica. Adicionalmente, con el fin de proporcionar una representación directa y detallada facilitando así la comprensión de este apartado, creé los diagramas de las dos arquitecturas de la red.

En cuanto al capítulo 4, para la parte destinada a la implementación de lo explicado en el capítulo anterior continué con la labor de documentación. En lo que a la explicación de la arquitectura del código se refiere, me dediqué a abstraer y formalizar la estructura del sistema y justificar el funcionamiento general de la aplicación y cómo estaba implementada.

El capítulo 5 se centra en el análisis de los modelos. Para ello, redacté el proceso de evaluación pertinente, realicé las diferentes validaciones y en cada modelo llevé a cabo un análisis detallado de los resultados obtenidos, discerniendo los datos relevantes y extrayendo conclusiones. Después, plasmé las ideas más importantes, descubrí un sesgo determinante en las métricas y observé qué características hacen que un modelo consiga resultados óptimos. Además, algunos de los resultados fueron distintos de los esperados, como el hecho de que los modelos complejos no consiguieran mejores resultados, por lo que ejecuté una labor de investigación y análisis minuciosos de las estructuras hasta dar con las respuestas.

Por último, cabe destacar el rol que adopté en la edición y corrección general de estilos en la memoria, buscando la coherencia visual y la integridad estructural del

documento.

En conclusión, mediante las tareas que he llevado a cabo en el proyecto he conseguido complementar la implementación de una arquitectura de *Deep Learning*, aportar en el diseño general de interfaces y flujo de la aplicación, realizar pruebas de modelos, validar la eficacia del método usado frente a alternativas tradicionales y he consolidado una labor de documentación exhaustiva del proyecto.

Bibliografía

- ALLEN, J. B. Short term spectral analysis, synthesis, and modification by discrete fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, Disponible en <https://ieeexplore.ieee.org/document/1162950>.
- ANDREA AGOSTINELLI, Z. B. J. E. M. V. A. C. Q. H. A. J. A. R. M. T. M. S. N. Z. C. F., TIMO I.DENK. Musiclm: Generating music from text. Disponible en <https://arxiv.org/pdf/2301.11325>.
- ANTHROPIC. Claude code docs: Descripción general. 2026.
- BARKAN, O., TSIRIS, D. y KOENIGSTEIN, N. Inversynth: Deep neural network for musical synthesizer parameter extraction. En *IEEE/ACM Transactions on Audio, Speech, and Language Processing*. 2019.
- CHOWNING, J. M. The synthesis of complex audio spectra by means of frequency modulation. *Journal of the Audio Engineering Society*, 1973.
- DAVIS, S. B. y MERMELSTEIN, P. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1980.
- DJEXPRESSIONS. Que es el adsr y como podemos aprovecharlo. Disponible en https://djexpressions.net/que-es-el-adsr-y-como-podemos-aprovecharlo/#google_vignette.
- DREAMSTIME. Más trucos para sintes old-school -técnicas de efectos y programación. Disponible en <https://www.futuremusic-es.com/mas-trucos-para-sintes-old-school-tecnicas-de-efectos-y-programacion/>.
- ENGEL, J., HANTRAKUL, L., GU, C. y ROBERTS, A. DDSP: Differentiable digital signal processing. En *International Conference on Learning Representations (ICLR)*. 2020.
- ENGEL, J., RESNICK, C., ROBERTS, A., SANDER, D., NOROUZI, M. y ECK, D. Neural audio synthesis of musical notes with wavenet autoencoders. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 2017.

- GIRSHICK, R. Fast R-CNN. En *Proceedings of the IEEE international conference on computer vision*, páginas 1440–1448. 2015.
- GONG, Y., CHUNG, Y.-A. y GLASS, J. AST: Audio spectrogram transformer. En *Proc. Interspeech 2021*. 2021.
- GREEN MUSICIANS. Dexed. Disponible en <https://www.greenmusicians.com/es/vst-gratuito-de-sintetizador/asb2m10/4459-asb2m10-dexed-3701701228270.html>.
- HISPASONIC. Yamaha dx7. Disponible en <https://www.hispasonic.com/productos/yamaha-dx7/19868>.
- HISPASÓNIC. Síntesis (5): síntesis aditiva. Disponible en <https://www.hispasonic.com/tutoriales/sintesis-5-sintesis-aditiva/38297>.
- HUBER, P. J. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, vol. 35(1), páginas 73–101, 1964.
- KILGOUR, K., ZULUAGA, M., ROBLEK, D. y SHARIF, M. Fréchet audio distance: A reference-free metric for evaluating music enhancement algorithms. Disponible en https://www.isca-archive.org/interspeech_2019/kilgour19_interspeech.pdf.
- KINGMA, D. P. y BA, J. Adam: A method for stochastic optimization. En *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. 2015.
- KONG, Q., CAO, Y., IQBAL, T., WANG, Y., WANG, W. y PLUMBLEY, M. D. Panns: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2020.
- KRIZHEVSKY, A., SUTSKEVER, I. y HINTON, G. E. Imagenet classification with deep convolutional neural networks. En *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2012.
- LECUN, Y., BOTTOU, L., BENGIO, Y. y HAFFNER, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.
- VAN DEN OORD, A., DIELEMAN, S., ZEN, H., SIMONYAN, K., VINYALS, O., GRAVES, A., KALCHBRENNER, N., SENIOR, A. y KAVUKCUOGLU, K. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.
- OPENCV. Convolutional neural network (cnn): A complete guide. Disponible en <https://developersbreach.com/convolution-neural-network-deep-learning/>.
- REVERB MACHINE. Aphex twin: Selected ambient works 85-92 synth sounds. Disponible en <https://reverbmachine.com/blog/aphex-twin-selected-ambient-works-85-92/>.

- STEVENS, S. S., VOLKMANN, J. y NEWMAN, E. B. A scale for the measurement of the psychological magnitude pitch. *The Journal of the Acoustical Society of America*, Disponible en <https://doi.org/10.1121/1.1915893>.
- VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, Ł. y POLOSUKHIN, I. Attention is all you need. En *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017.
- WIKIPEDIA. Audio bit depth. Disponible en https://en.wikipedia.org/wiki/Audio_bit_depth.
- WIKIPEDIA. Espectrograma. Disponible en <https://es.wikipedia.org/wiki/Espectrograma>.
- WIKIPEDIA. Síntesis sustractiva. Disponible en https://es.wikipedia.org/wiki/S%C3%ADntesis_sustractiva.
- YEE-KING, M., FEDDEN, L. y DÍNVERNO, M. Automatic programming of VST sound synthesizers using deep networks and other techniques. *IEEE Transactions on Emerging Topics in Computational Intelligence*, Disponible en <https://doi.org/10.1109/TETCI.2017.2785209>.

